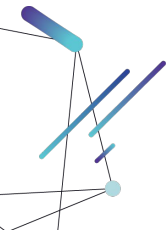


Vision Transformers (ViT)

Tatiana Gaintseva

План

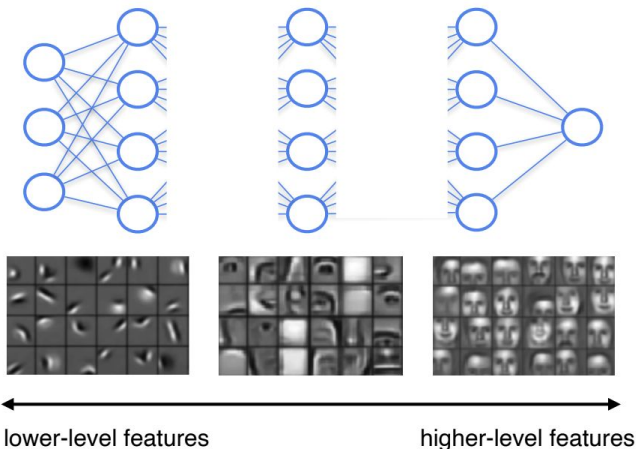
- Зачем вообще Transformers для картинок, если есть CNN?
- Структура ViT
- Что ViT выучивают
- Разные варианты ViT и ViT+CNN (Swin, ConvNeXT, etc)
- ViT как универсальный backbone



CNNs -> Vision Transformers

CNN достаточно хорошо справляются с обработкой картинок. Во многом это из-за хороших inductive biases:

- **Locality:** фильтры обрабатывает небольшую локальную область картинки
- **Translation equivariance → weight sharing:** все части картинки обрабатываются одним и тем же механизмом $f(Tx) = Tf(x)$
- **Approximate translation invariance:** из-за pooling слоев небольшие изменения положения объекта на картинке практически не меняют ответ сети
- **Hierarchical compositionality:** первые слои реагируют на низкоуровневые, локальные паттерны. Следующие слои собирают из этой информации более высокоуровневые представления.



pixels → edges → textures → parts → objects

CNNs -> Vision Transformers

Почему хочется адаптировать Transformers для Vision:

- Из-за **Hierarchical compositionality** слои начинают видеть "global context" только на последних слоях. Attention у Трансформеров позволяет видеть глобальный контекст сразу.
- **Transformers** показали себя очень хорошо в NLP. Transformers для Vision – шаг к унификации доменов NLP и CV

Давайте соберем ViT для задачи классификации картинок,
как это сделали в первой статье про ViT

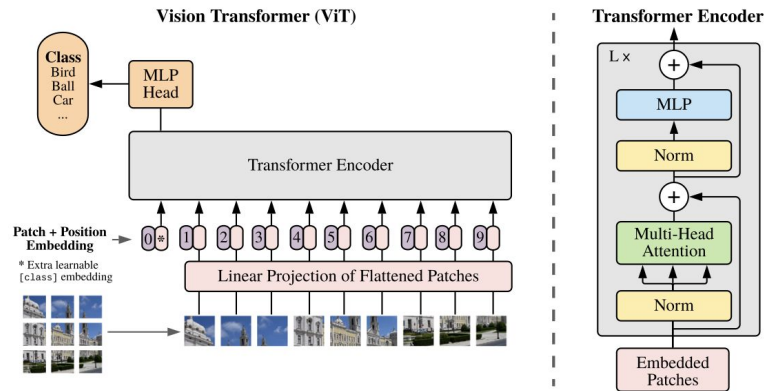
AN IMAGE IS WORTH 16X16 WORDS: TRANSFORMERS FOR IMAGE RECOGNITION AT SCALE

Alexey Dosovitskiy^{*,†}, Lucas Beyer^{*}, Alexander Kolesnikov^{*}, Dirk Weissenborn^{*},
Xiaohua Zhai^{*}, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer,
Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, Neil Houlsby^{*,†}

^{*}equal technical contribution, [†]equal advising

Google Research, Brain Team

{adosovitskiy, neilhoulbsy}@google.com



Архитектура ViT

ViT

Давайте соберем ViT для задачи классификации картинок, как это сделали в первой статье про ViT

Главные вопросы:

- Transformer работает с токенами, а у нас картинка.

Как получить токены из картинки?

- Transformer использует `positional embeddings`.

Какие должны быть `positional embeddings` в случае картинки?

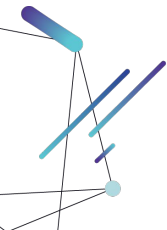


Image Tokens

1. Делим картинку на $k \times k$ патчей размера $p \times p$.
У картинки с цветовых каналов

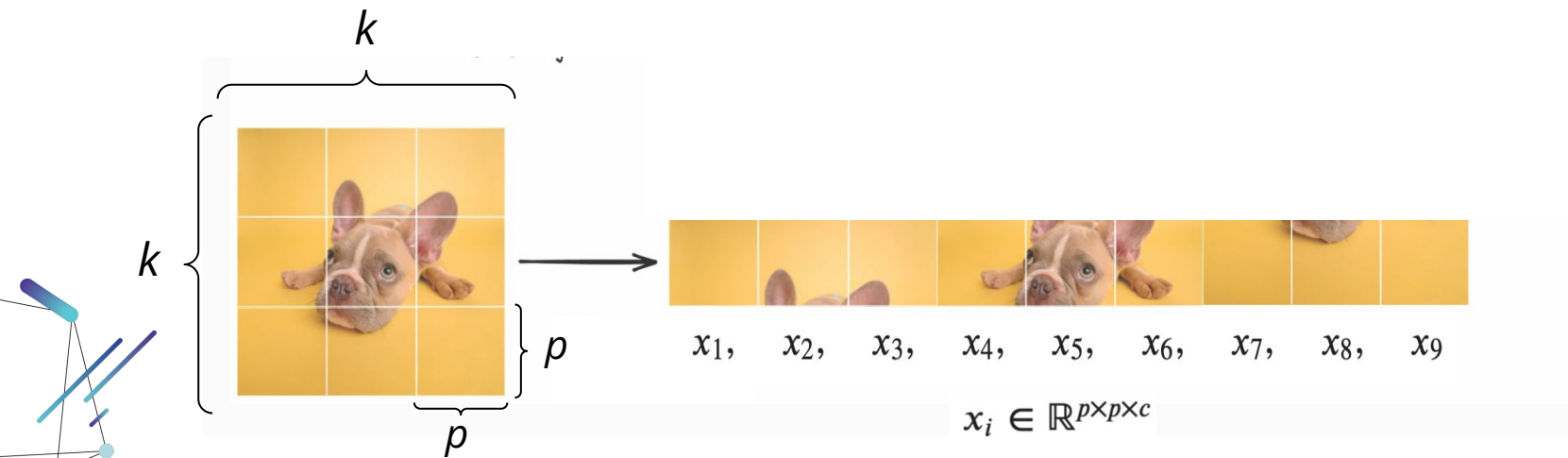


Image Tokens

2. Разворачиваем каждый патч в вектор

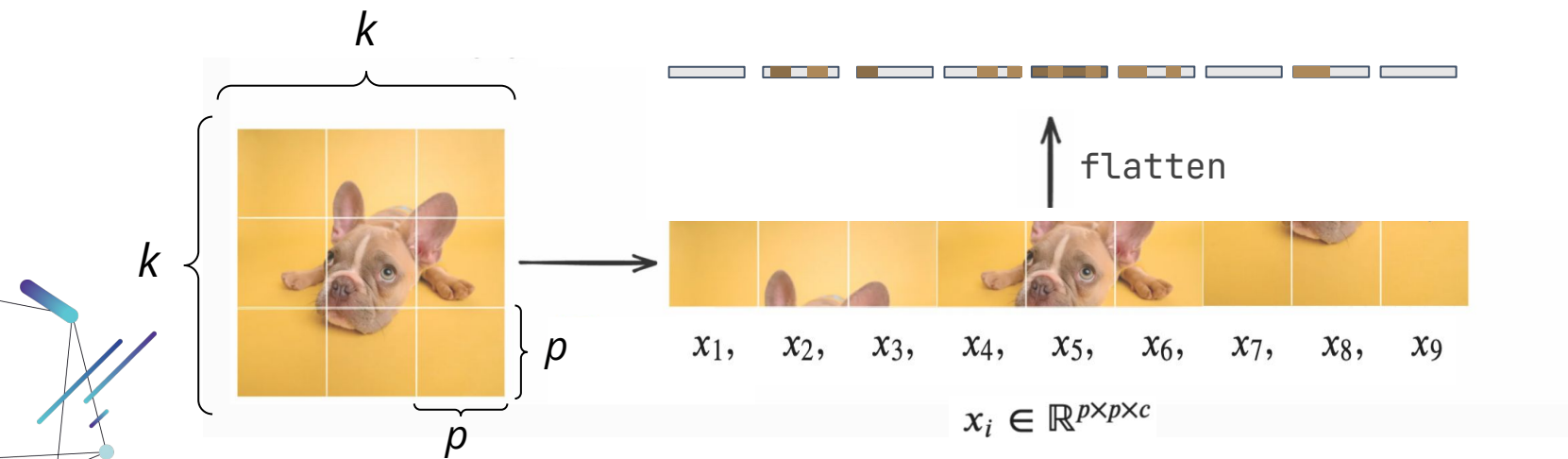


Image Tokens

3. Прогоняем через линейный слой, получаем эмбединги патчей

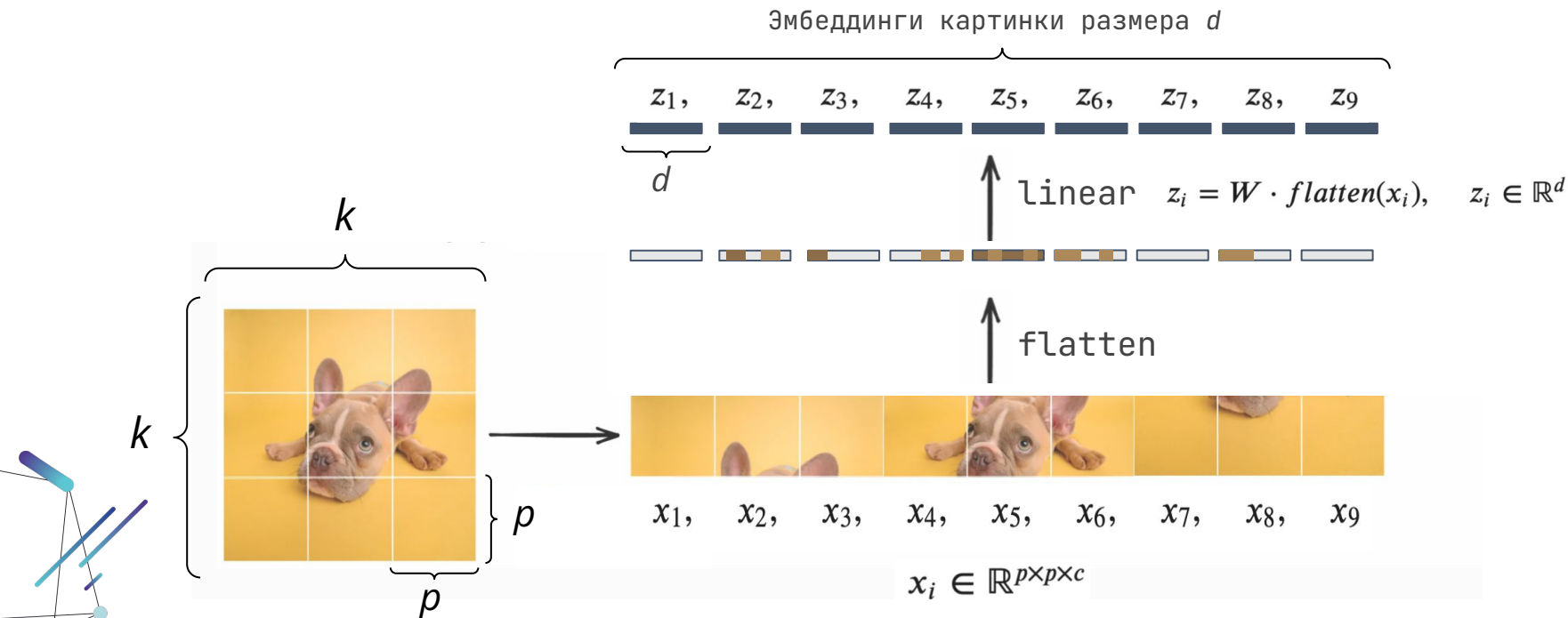


Image Tokens

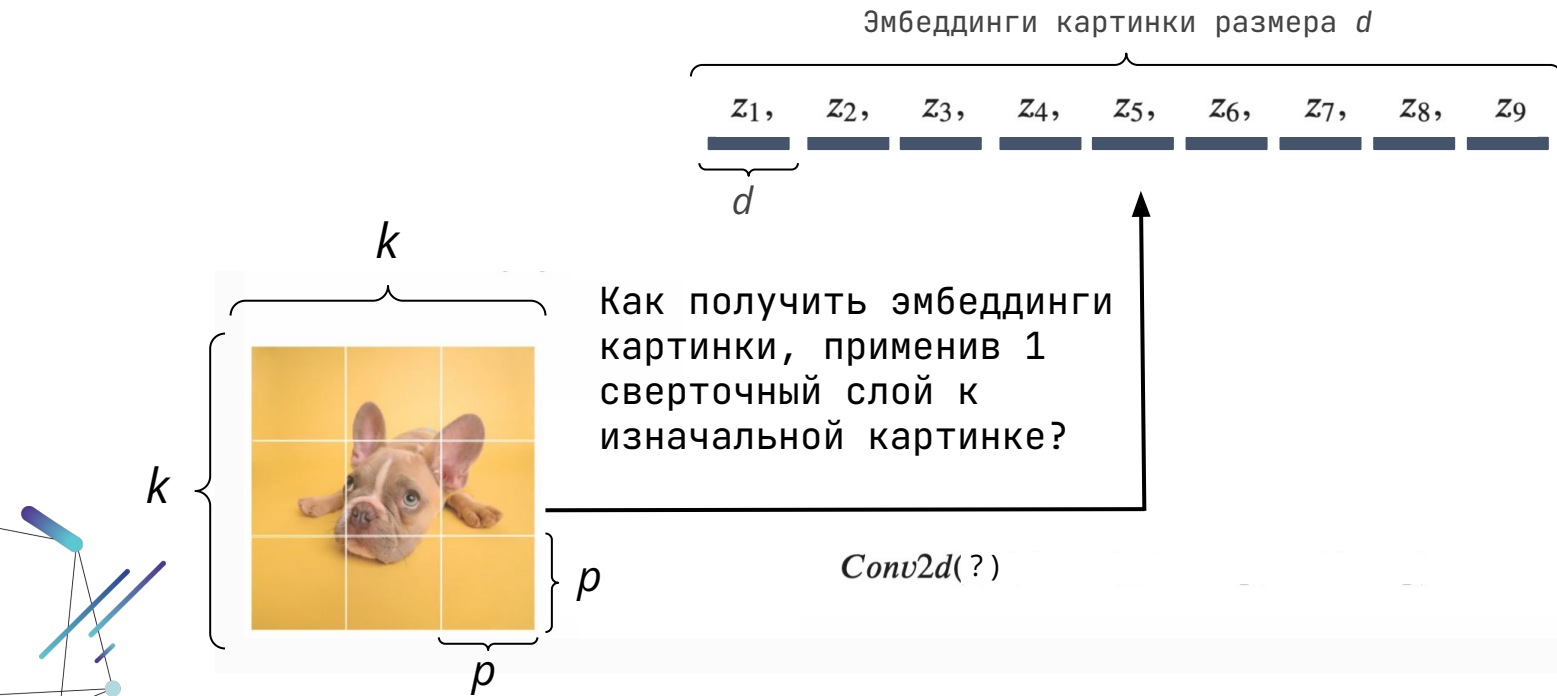


Image Tokens

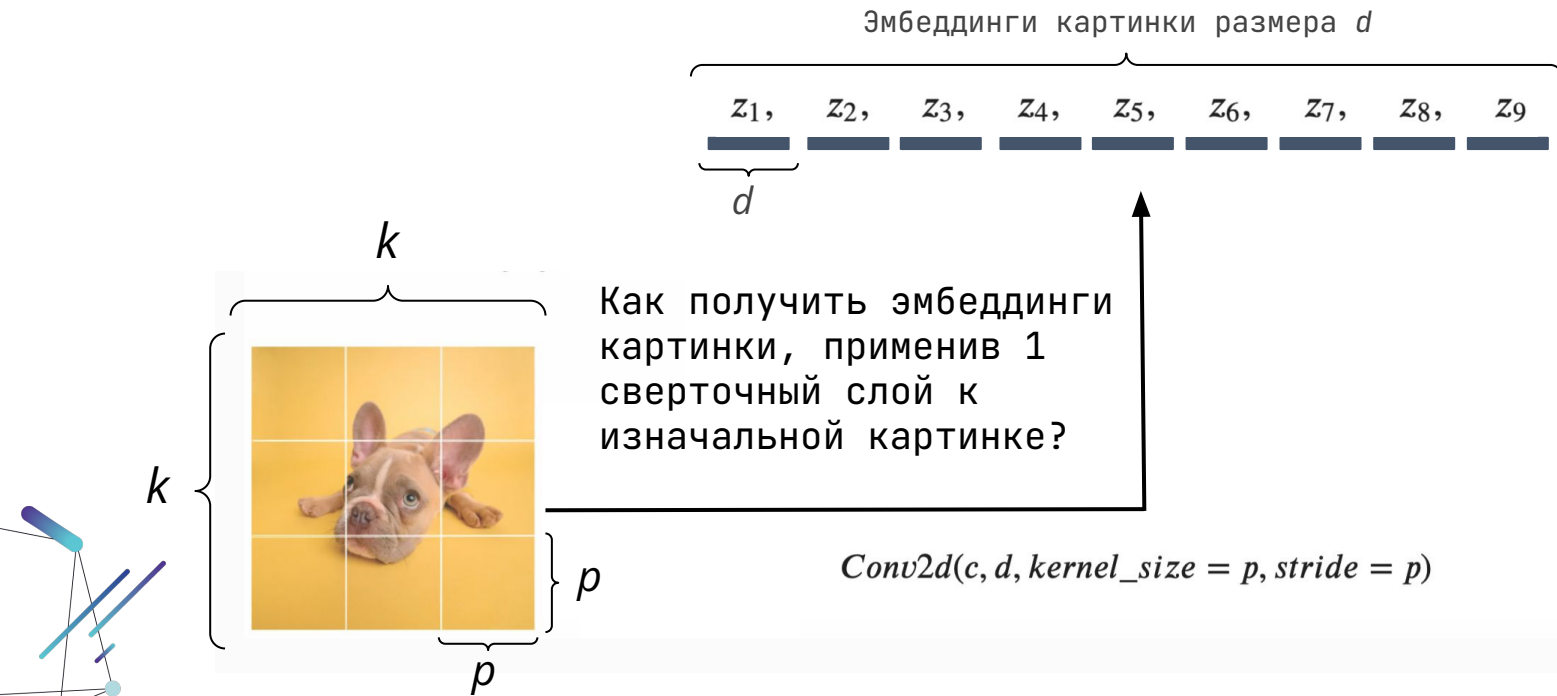
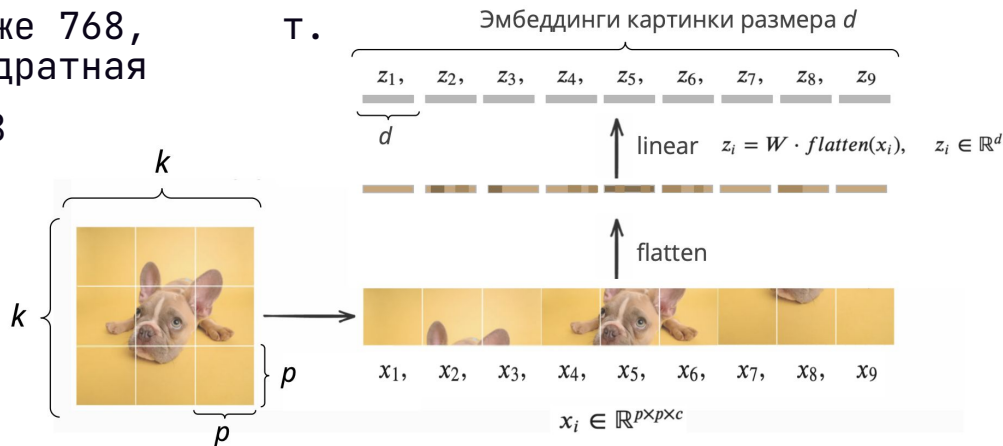


Image Tokens

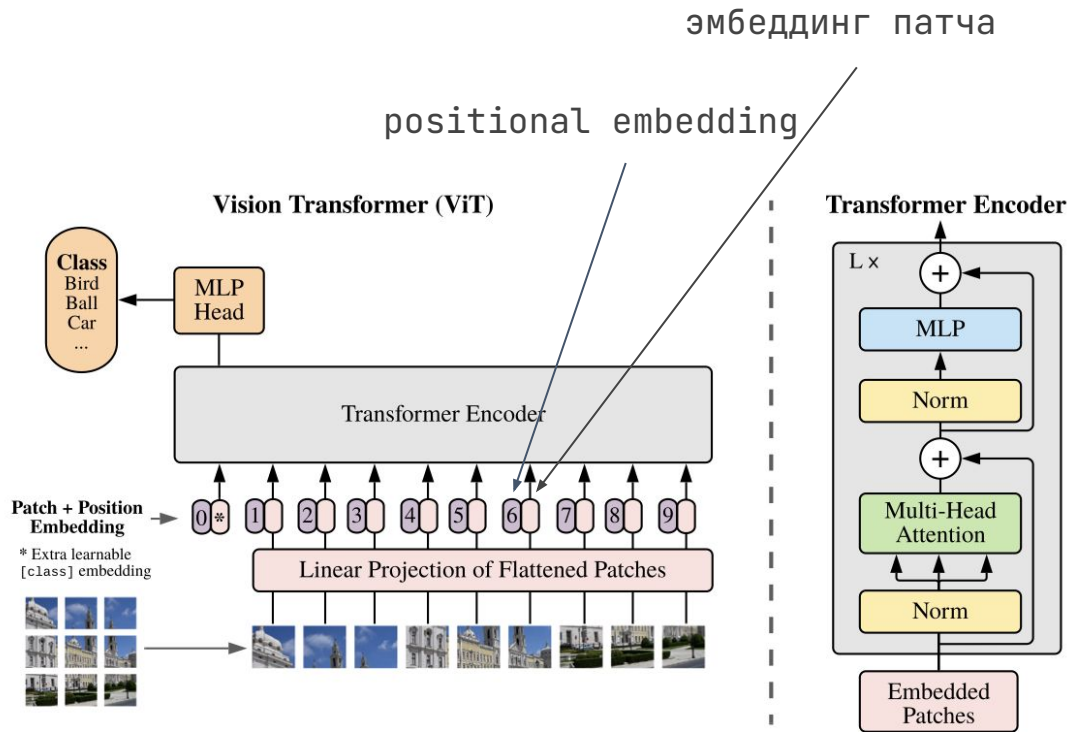
Обычные значения как у [google/vit-large-patch16-224](https://research.google.com/publications/vit-large-patch16-224/):

- Входная картинка размера $224 \times 224 \times 3$
- Каждый патч размера $p=16 \times 16$
- Всего патчей $k \times k=14 \times 14=196$
- Размер патча после flatten $16 \times 16 \times 3=768$
- Размер эмбединга картинки тоже 768,
е. матрица линейного слоя квадратная

Итого: 196 токенов размерности 768



ViT

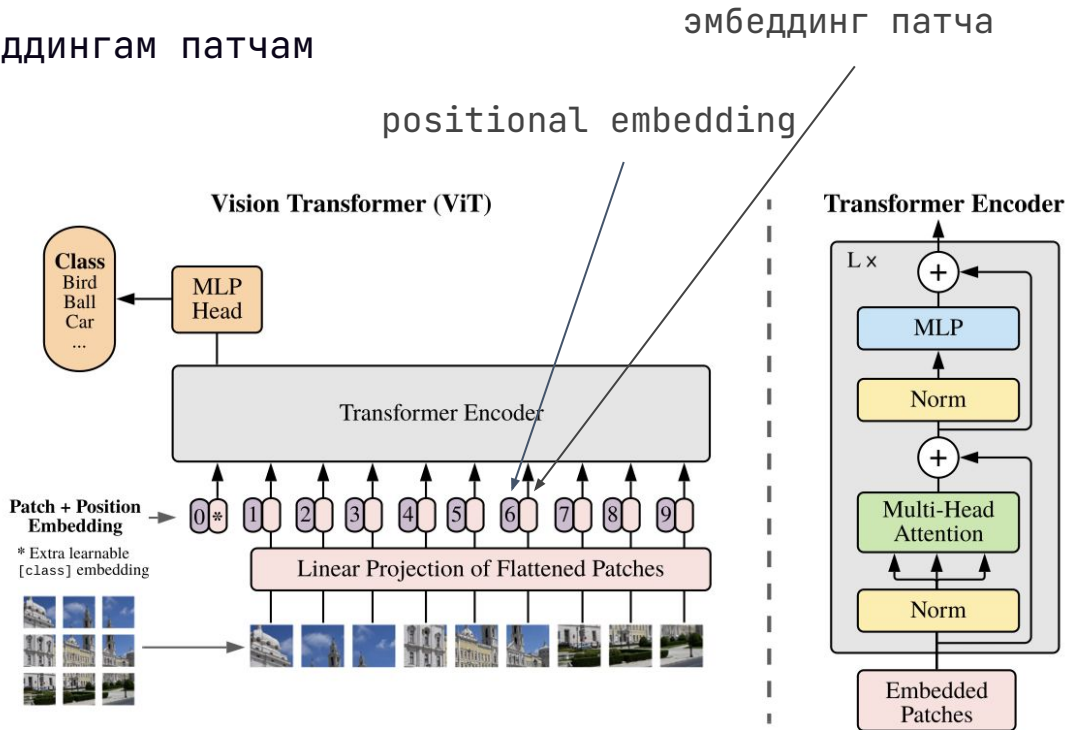


ViT positional embeddings

Теперь нам надо добавить к эмбедингам патчам positional embeddings.

Идеи:

- Как в NLP, **1D-positional embeddings** соответствующие номеру патча (learnable/frozen)
- **2D-positional embedding** (соответствующие номеру строки и столбца), (learnable/frozen)



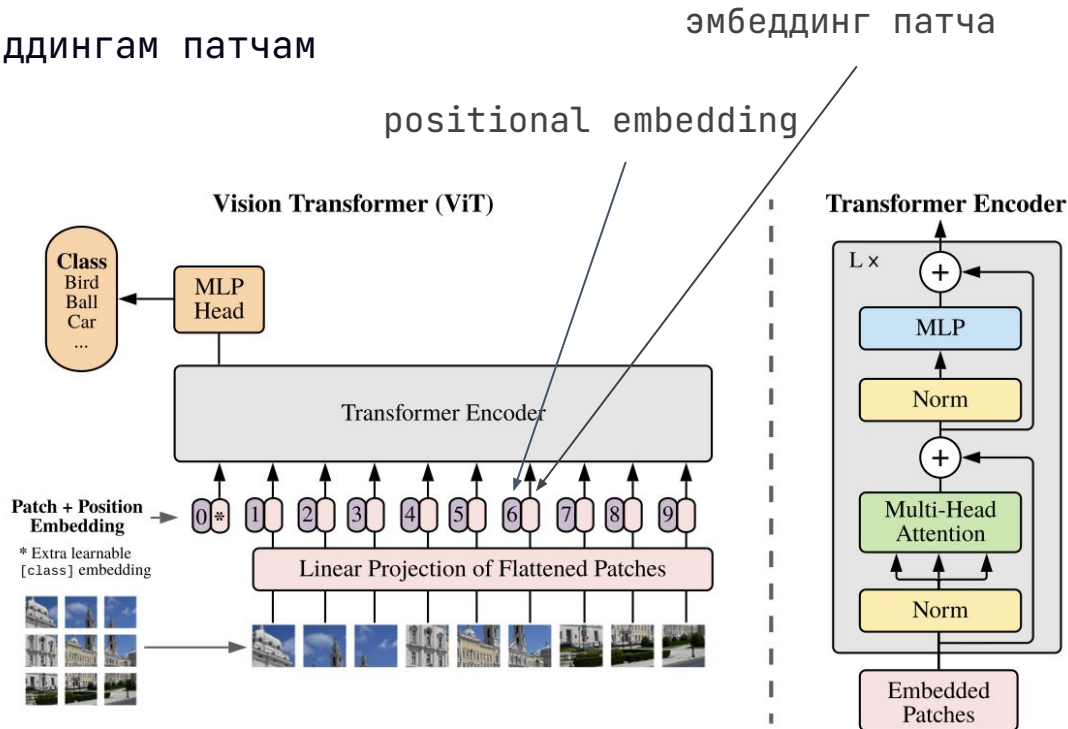
ViT positional embeddings

Теперь нам надо добавить к эмбедингам патчам positional embeddings.

Идеи:

- Как в NLP, 1D-positional embeddings соответствующие номеру патча (learnable)

Это сработало лучше

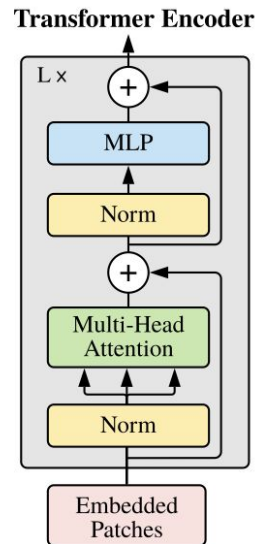
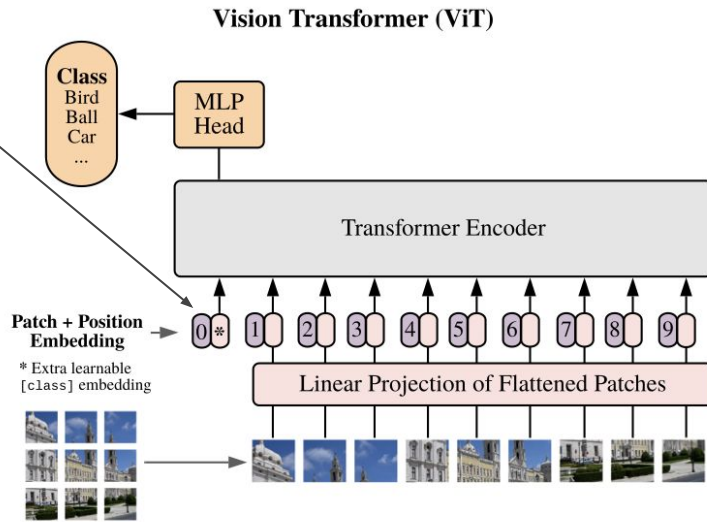


ViT CLS token

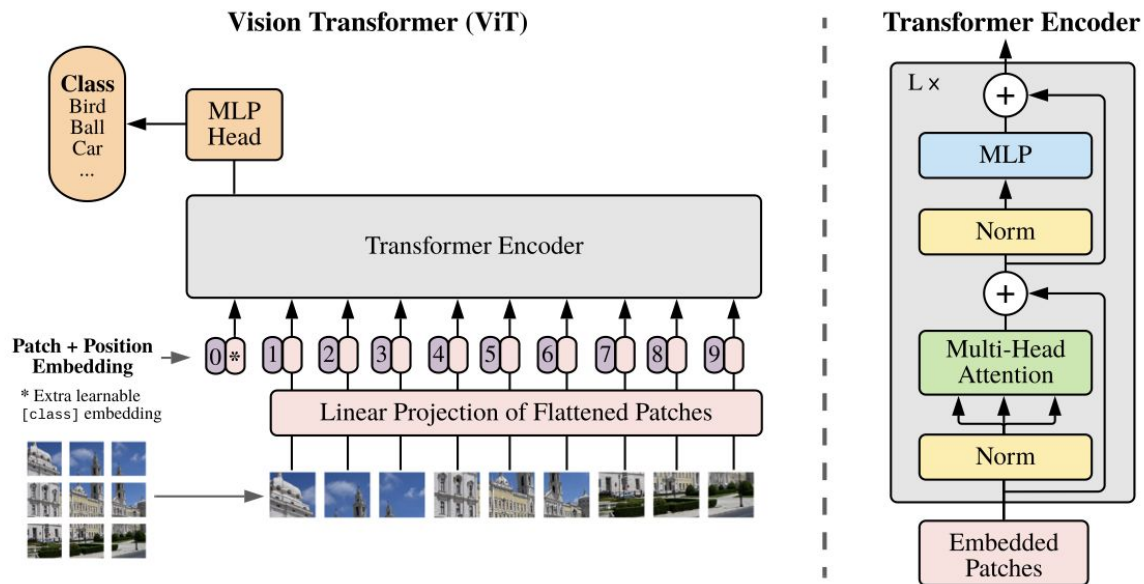
Добавляем еще один эмбединг,
соответствующий [CLS] token
(как в BERT)

Он trainable,
инициализируется случайным
шумом

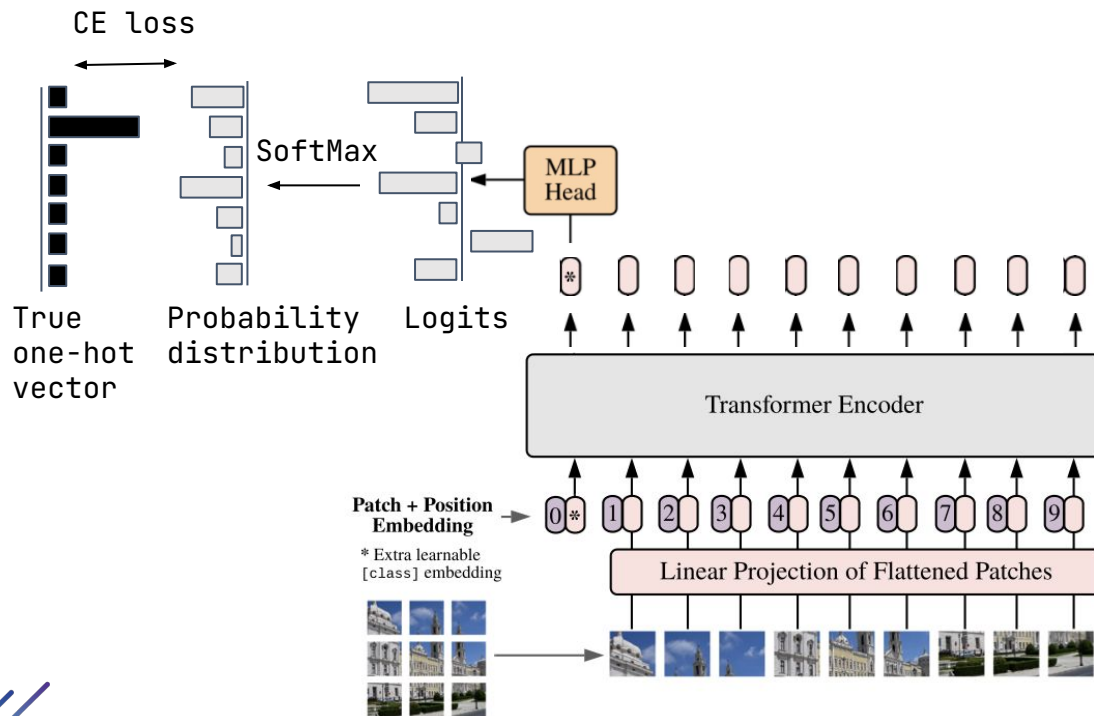
Размерность входа в
Transformer становится 197x768



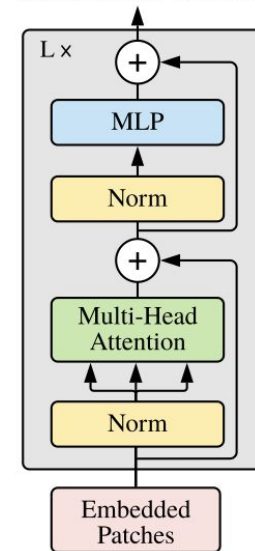
ViT архитектура



ViT архитектура



Transformer Encoder



ViT vs CNN

У ViT меньше архитектурных inductive bias

Prior	CNN	ViT
Locality (фокус на соседние пиксели)	✓ С помощью фильтров небольшого размера	✗ каждый патч смотрит на каждый другой патч через self-attention
Translation equivariance (все части картинки обрабатываются одним и тем же механизмом)	✓ Каждый фильтр обрабатывает все части картинки одинаково	✗ из устройства архитектуры
Hierarchy	✓ reception fields, pooling	✗ resolution всегда одинаковый

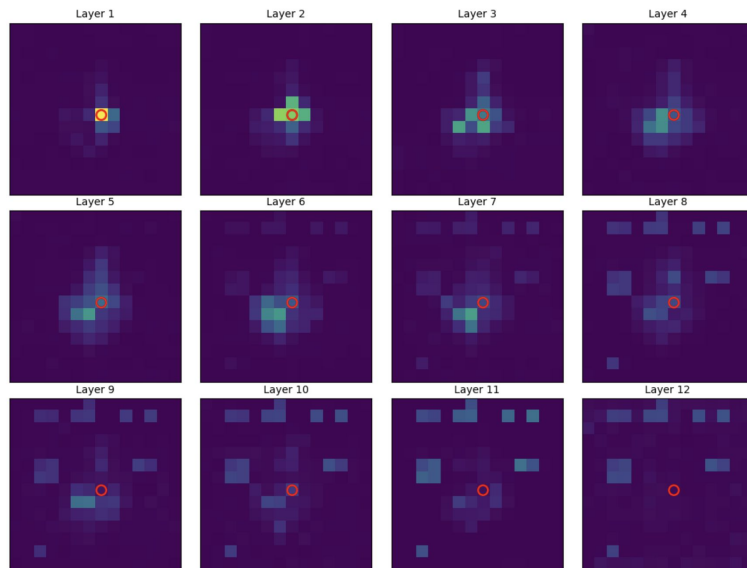
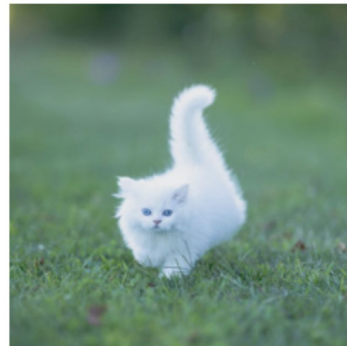
Что выучивают ViT

Что мы поняли про “локальность” ViT vs CNN:

- CNN: первые слои смотрят на “локальные” области картинки, receptive field растет к концу сети
- ViT по определению всегда смотрят на картинку глобально

На практике:

- Первые слои ViT выучивают в основном локальные паттерны внимания
- В последних слоях сети attention паттерны уже глобальные.



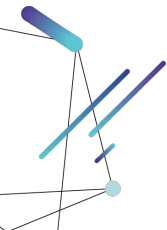
ViT vs CNN

В целом, как можно дизайннить и учить ML-алгоритмы:

1. Добавить некоторую информацию про домен/данные в саму архитектуру модели.
Остальную информацию модель выучит из данных
2. Оставить в архитектуре как можно меньше *inductive biases* –
пусть модель выучивает все из данных

Второй подход обычно требует больше (и более разнообразных) данных для обучения.
Но если эти данные есть, подход 2 может оказаться лучше по качеству:

- Модель сама понимает на основе данных, какие *inductive bias* ей действительно нужны, чтобы решить задачу
- Некоторые *inductive biases* легче выучить, чем захардкодить

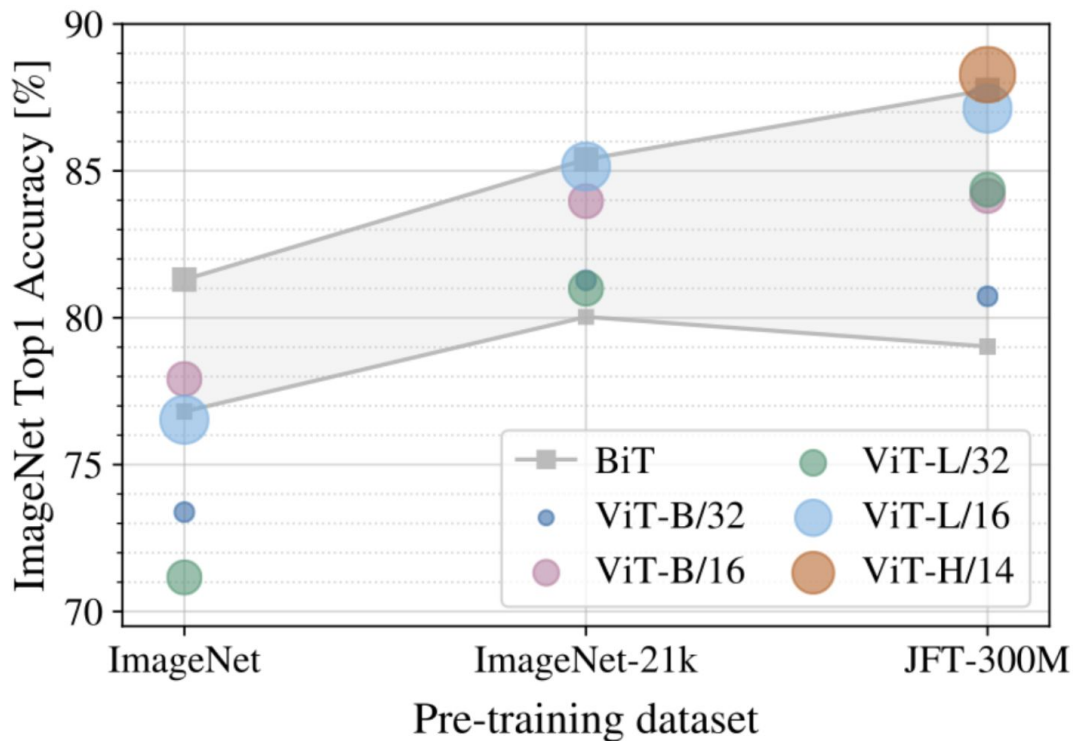


ViT vs CNN

Мало данных →
CNN лучше

Много данных
→ ViT лучше

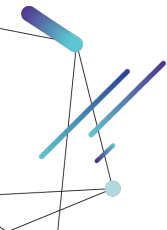
Чем больше данных,
тем лучше метрики
у больших ViT



ViT как универсальный backbone

ViT — дефолтный backbone во многих Vision models:

- ViT для сегментации/детекции
- CLIP
- DINO/DINOv2
- SAM
- LLaVA, GPT-4V, multimodal LLMs
- Stable Diffusion, SANA, FLUX

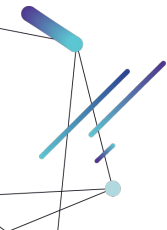


Итоги видео

В этом видео мы:

- Поняли мотивацию адаптировать ViT для задач компьютерного зрения
- Разобрали классическую архитектуру ViT для задачи классификации картинок
- Обсудили различие inductive biases CNN и ViT, и что это значит на практике

В следующем видео мы поговорим про разные вариации ViT и смеси ViT+CNN



Архитектуры Vision Transformers (ViT) и паттерны attention

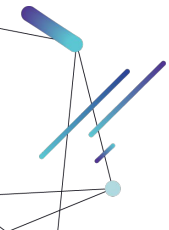
Tatiana Gaintseva

План

В этом видео мы обсудим различные архитектуры ViT и ViT+Conv и варианты Attention паттернов.

Архитектуры:

- DeiT
- Swin Transformer (... PVT, MViT)
- CvT (... CoAtNet, MaxViT)
- ConvNeXT
- MetaFormer / PoolFormer



DeiT

DeiT – Vision Transformer, предобученный на ImageNet-1k и по качеству схожий с CNN.

Архитектурных изменений по сравнению с ViT нет.

Трюки:

- Очень много аугментаций
- Сильная аккуратно затюненная регуляризация

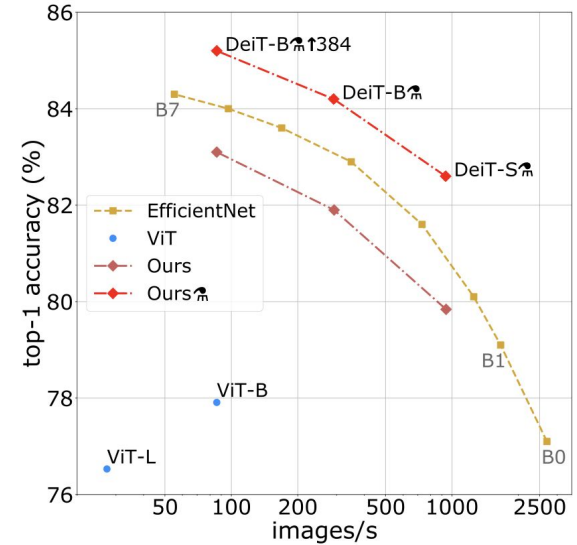


Figure 1: Throughput and accuracy on Imagenet of our methods compared to EfficientNets, trained on Imagenet1k only. The throughput is measured as the number of images processed per second on a V100 GPU. DeiT-B is identical to ViT-B, but the training is more adapted to a data-starving regime. It is learned in a few days on one machine. The symbol d refers to models trained with our transformer-specific distillation. See Table 5 for details and more models.

DeiT

DeiT – Vision Transformer, предобученный на ImageNet-1k и по качеству схожий с CNN.

Архитектурных изменений по сравнению с ViT нет.

Трюки:

- Очень много аугментаций
- Сильная аккуратно затюненная регуляризация
- Дистилляция из CNN teacher model (RegNetY)

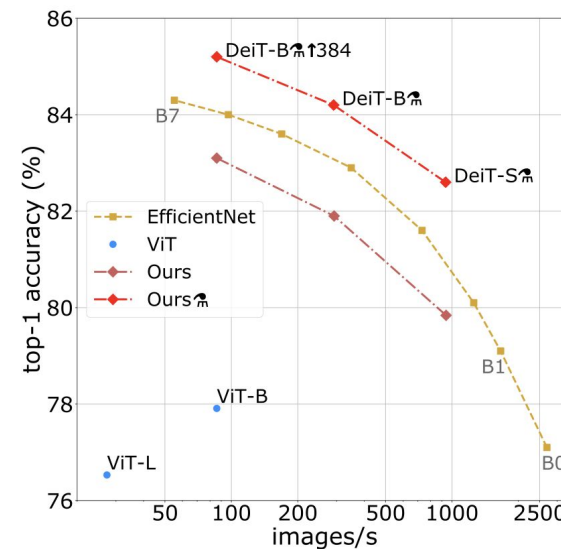


Figure 1: Throughput and accuracy on Imagenet of our methods compared to EfficientNets, trained on Imagenet1k only. The throughput is measured as the number of images processed per second on a V100 GPU. DeiT-B is identical to ViT-B, but the training is more adapted to a data-starving regime. It is learned in a few days on one machine. The symbol d refers to models trained with our transformer-specific distillation. See Table 5 for details and more models.

DeiT: дистилляция

Лосс-функция при дистилляции:

$$\mathcal{L}_{\text{global}}^{\text{hardDistill}} = \frac{1}{2} \mathcal{L}_{\text{CE}}(\psi(Z_s), y) + \frac{1}{2} \mathcal{L}_{\text{CE}}(\psi(Z_s), y_t).$$

Обычный лосс между
ответами сети
и правильными лейблами

Лосс между ответами
сети
и ответами сети-учителя

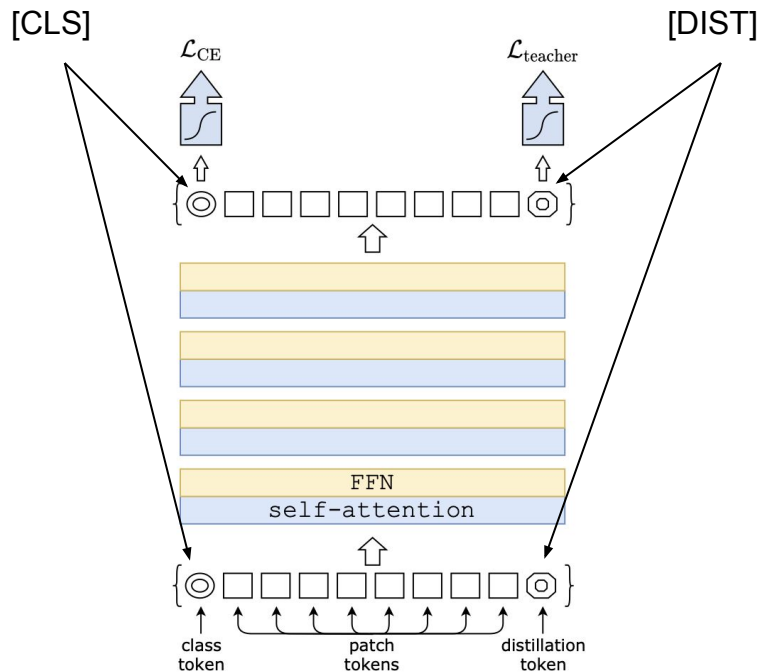


Figure 2: Our distillation procedure: we simply include a new *distillation token*. It interacts with the class and patch tokens through the self-attention layers. This distillation token is employed in a similar fashion as the class token, except that on output of the network its objective is to reproduce the (hard) label predicted by the teacher, instead of true label. Both the class and distillation tokens input to the transformers are learned by back-propagation.

DeiT

DeiT – Vision Transformer, предобученный на ImageNet-1k и по качеству схожий с CNN.

Архитектурных изменений по сравнению с ViT нет.

Трюки:

- Очень много аугментаций
- Сильная аккуратно затюненая регуляризация

Вывод: если сместить inductive biases в сторону стратегии обучения, ViT могут обучиться так же хорошо, как CNN, на малом количестве данных

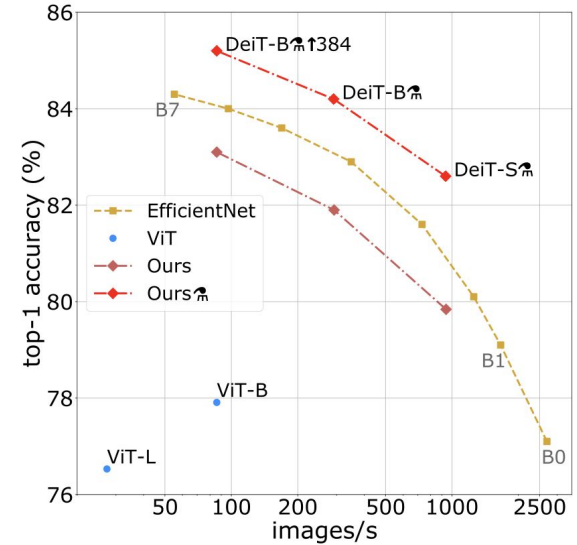


Figure 1: Throughput and accuracy on Imagenet of our methods compared to EfficientNets, trained on Imagenet1k only. The throughput is measured as the number of images processed per second on a V100 GPU. DeiT-B is identical to ViT-B, but the training is more adapted to a data-starving regime. It is learned in a few days on one machine. The symbol d refers to models trained with our transformer-specific distillation. See Table 5 for details and more models.

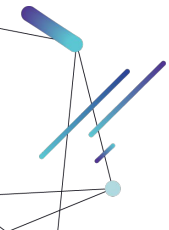
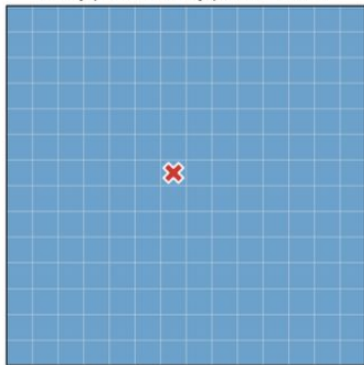
Варианты ViT Attention

Стандартный вариант Attention – dense, плотный.
Каждый патч смотрит на каждый другой.

Это вычислительно дорого: $O(n^2)$, где n – количество токенов

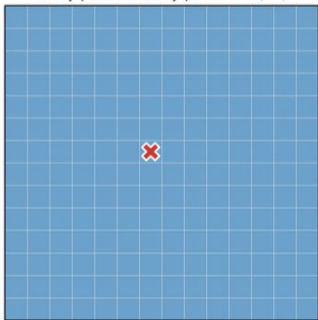
Что бы такого придумать, чтобы было дешевле?

Full attention (ViT)
every patch ↔ every patch — $O(N^2)$

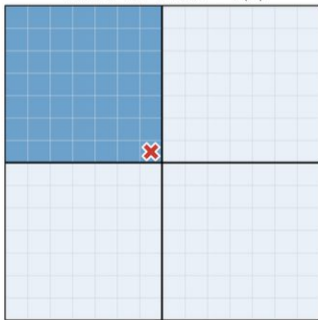


Варианты ViT Attention

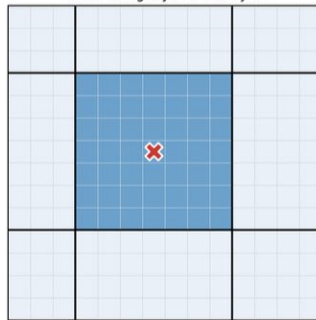
Full attention (ViT)
every patch $\leftrightarrow$ every patch — $O(N^2)$



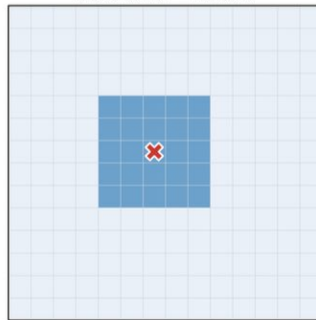
Window (Swin)
within a 7×7 window — $O(N)$



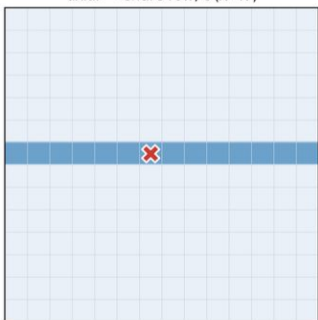
Shifted window (Swin)
alternating layer shifts by 3



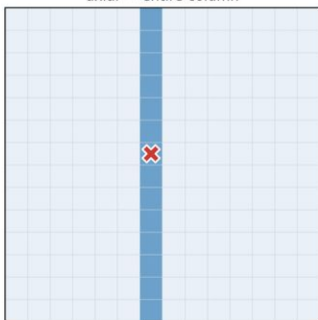
Neighborhood (NAT)
a 5×5 kernel — like conv



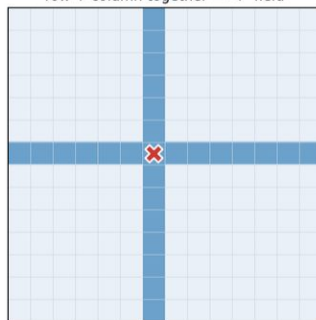
Row attention
axial — entire row, $O(N\sqrt{N})$



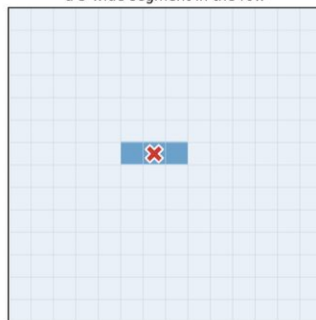
Column attention
axial — entire column



Cross-shape (CSWin / CCNet)
row + column together — '+' field

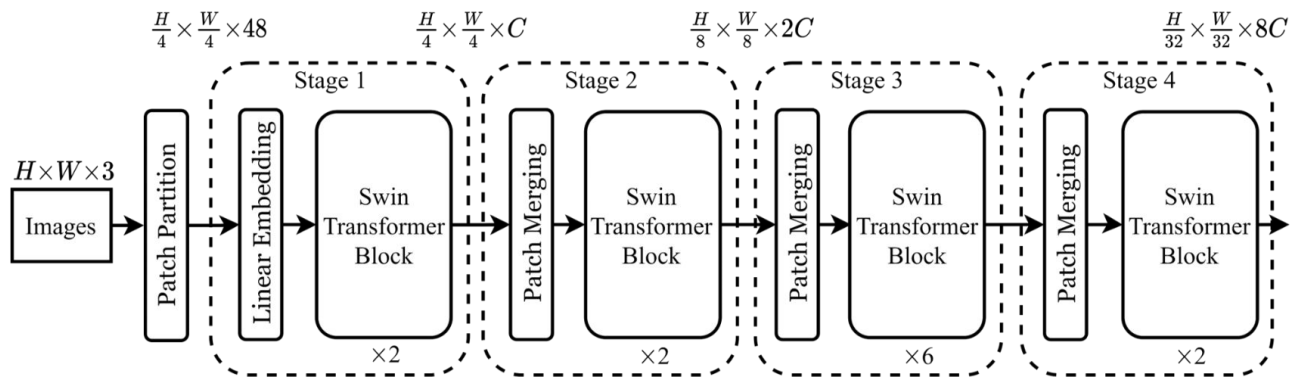


1D row strip
a 3-wide segment in the row



red x = query patch ; dark blue = patches the query can attend to

Swin (Shifted Windows) Transformer

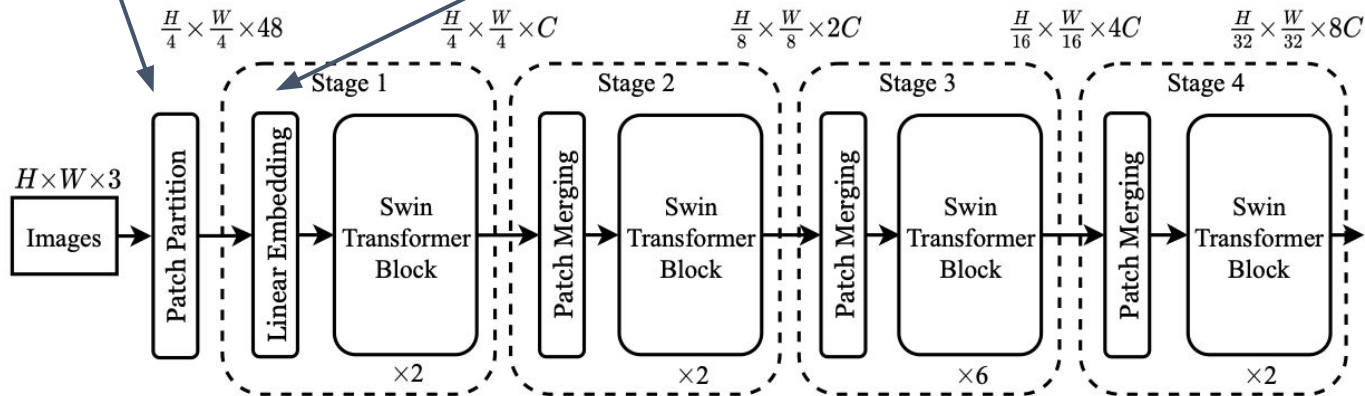


(d) Architecture

Swin Transformer

Делим изображение на токены размера 4×4 ,
получая $\frac{H}{4} \times \frac{W}{4}$ патчей размера $4 \times 4 \times 3 = 48$

Каждый патч эмбедрится в
вектор размера C

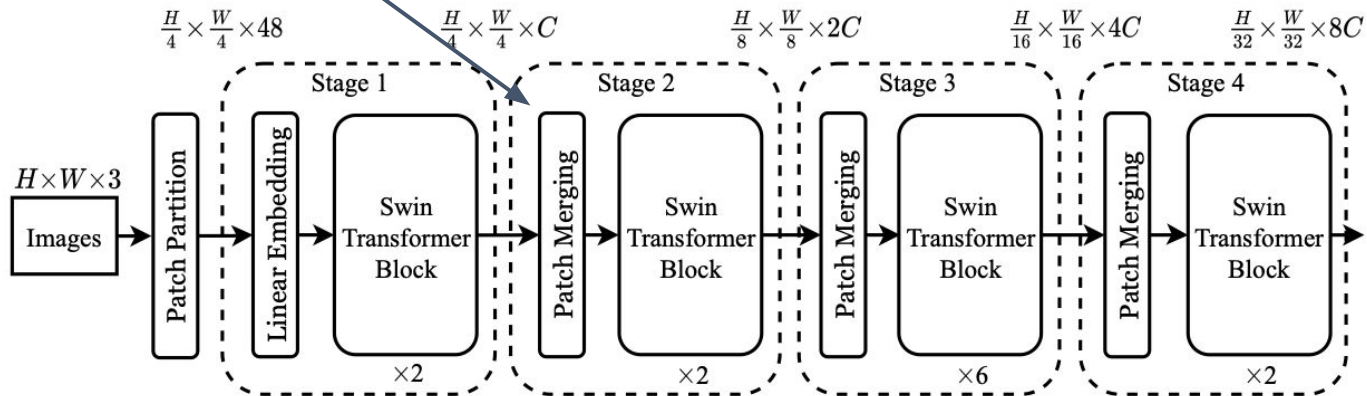
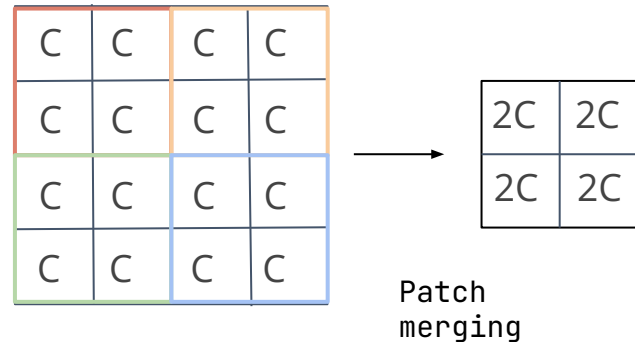


Swin Transformer

Делим патчи на квадраты 2×2 , мерджим эмбединги патчей внутри каждого квадрата в один эмбединг размера $2C$:

$$emb^{stage2} = W_{4C \times 2C} [emb_1^{stage1}, emb_2^{stage1}, emb_3^{stage1}, emb_4^{stage1}]$$

Получаем $\frac{H}{8} \times \frac{W}{8}$ патчей

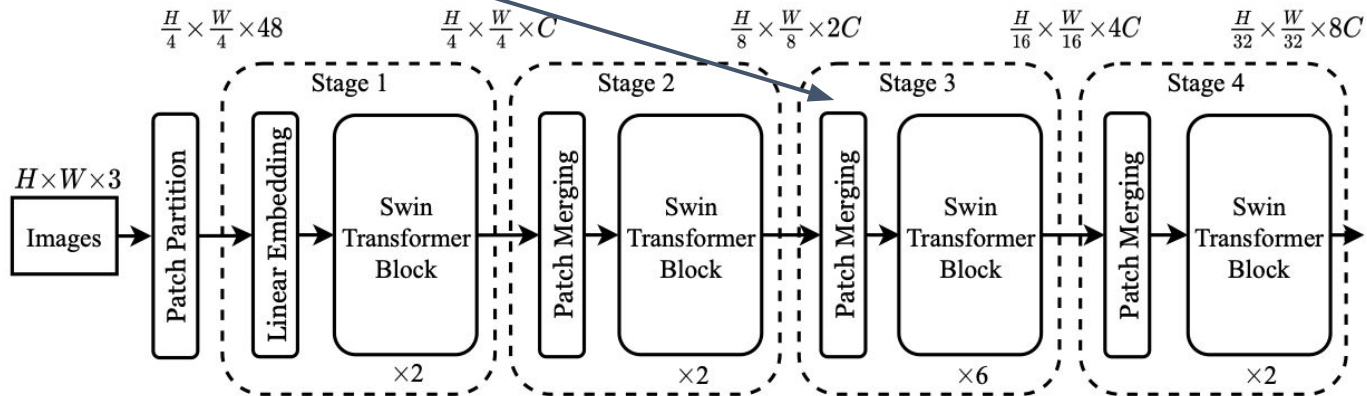
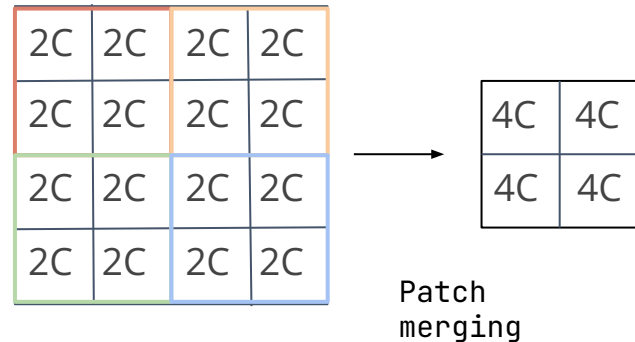


Swin Transformer

Делим патчи на квадраты 2×2 , мерджим эмбединги патчей внутри каждого квадрата в один эмбединг размера $4C$:

$$emb^{stage3} = W_{8C \times 4C} [emb_1^{stage2}, emb_2^{stage2}, emb_3^{stage2}, emb_4^{stage2}]$$

Получаем $\frac{H}{16} \times \frac{W}{16}$ патчей

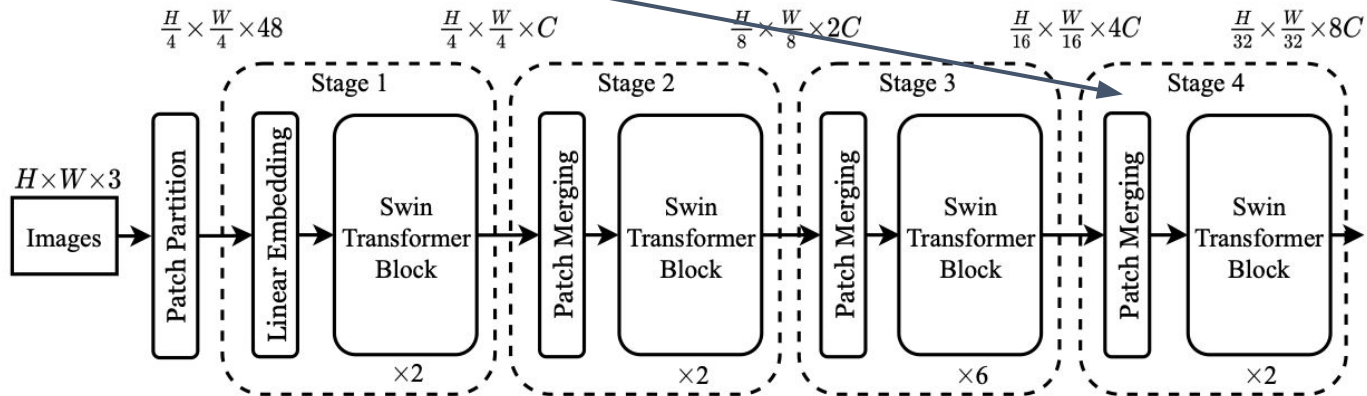
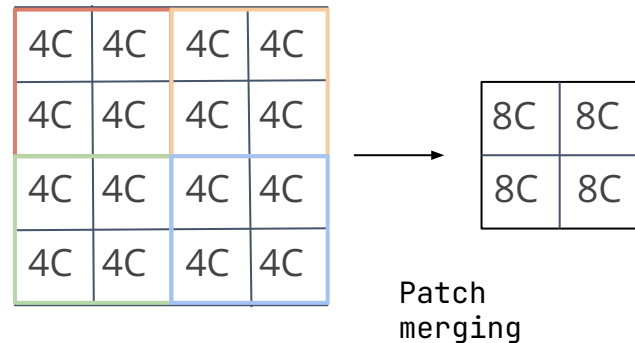


Swin Transformer

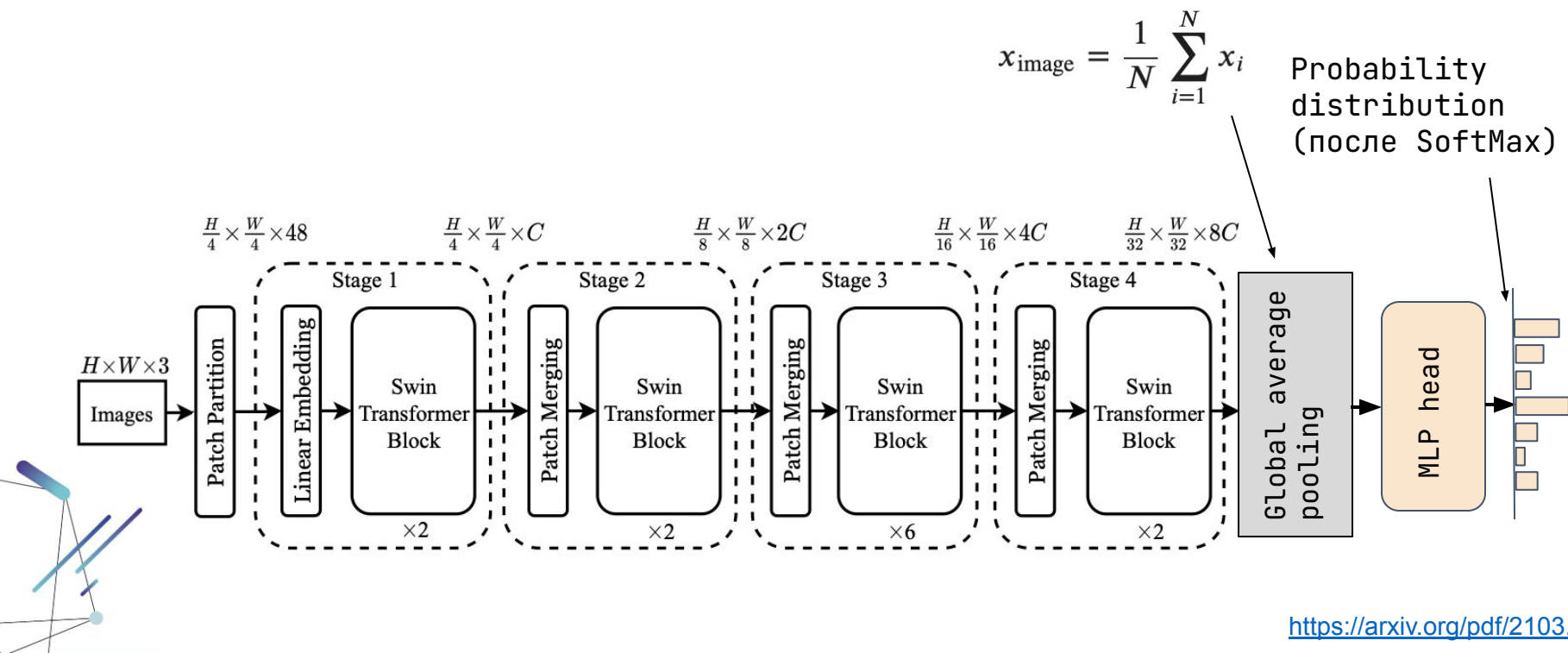
Делим патчи на квадраты 2×2 , мерджим эмбединги патчей внутри каждого квадрата в один эмбединг размера $4C$:

$$emb^{stage4} = W_{16C \times 8C} [emb_1^{stage3}, emb_2^{stage3}, emb_3^{stage3}, emb_4^{stage3}]$$

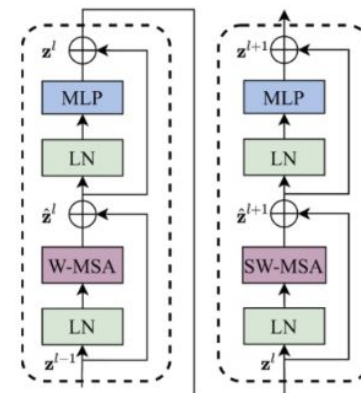
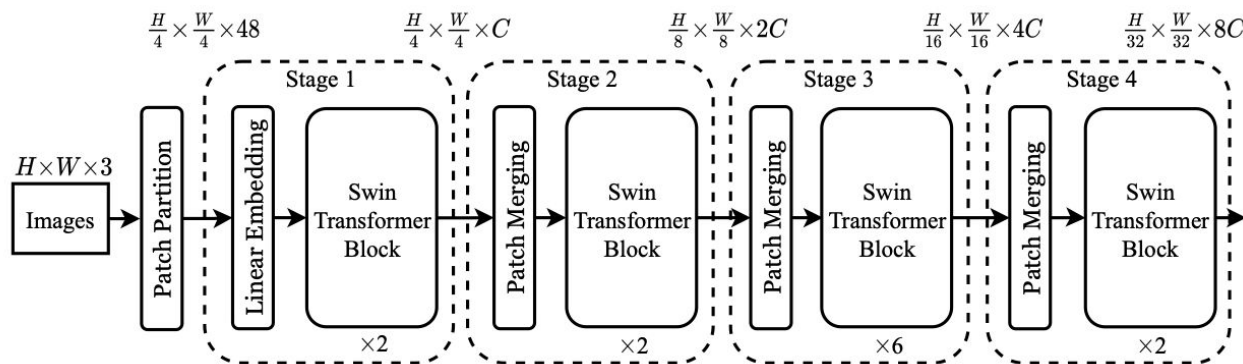
Получаем $\frac{H}{32} \times \frac{W}{32}$ патчей



Swin (Shifted Windows) Transformer

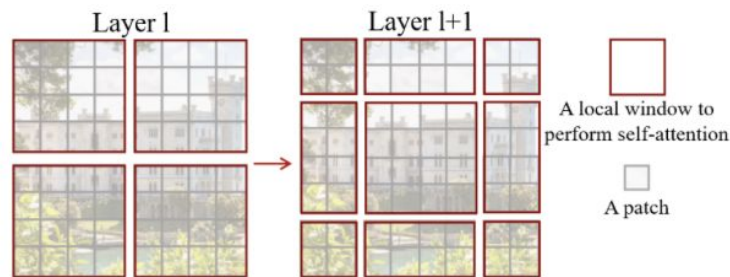


Swin Transformer

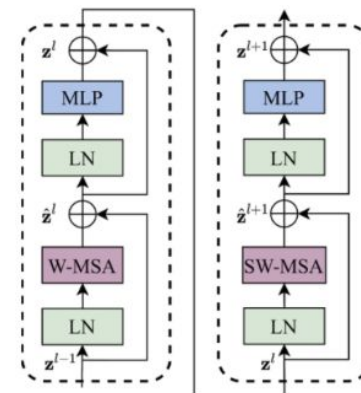


(c) Two Successive Swin Transformer Blocks

Swin Transformer

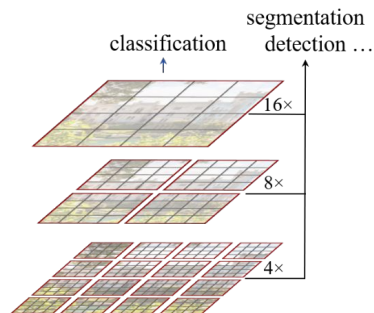


(b) Shifted Window

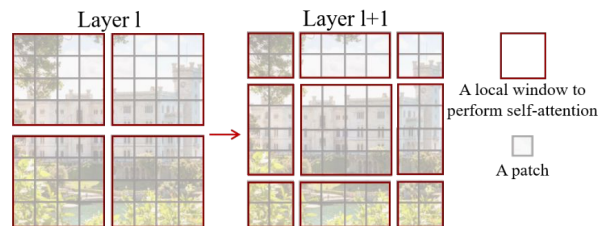


(c) Two Successive Swin Transformer Blocks

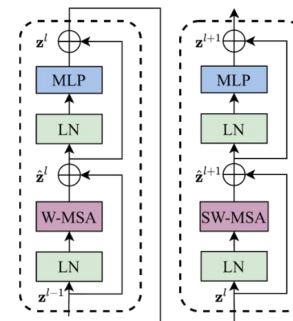
Swin Transformer



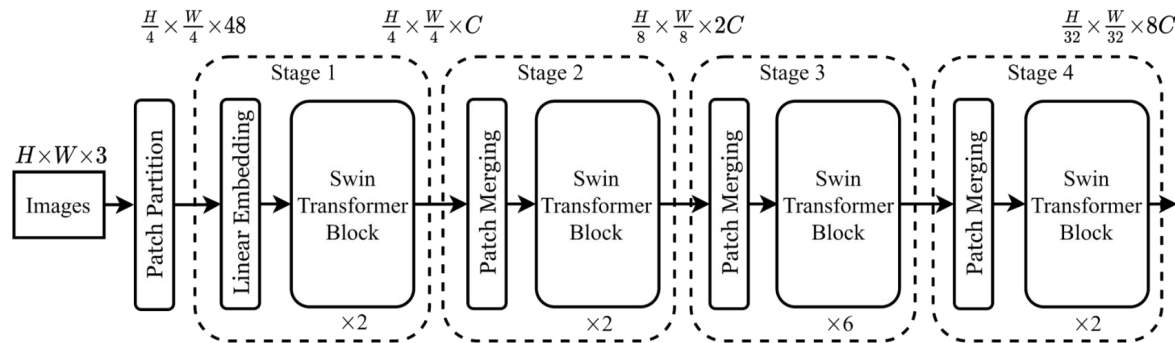
(a) Swin Transformer



(b) Shifted Window



(c) Two Successive Swin Transformer Blocks



(d) Architecture

Swin Transformer

Результаты экспериментов:

(a) Regular ImageNet-1K trained models					
method	image size	#param.	FLOPs	throughput (image / s)	ImageNet top-1 acc.
RegNetY-4G [48]	224 ²	21M	4.0G	1156.7	80.0
RegNetY-8G [48]	224 ²	39M	8.0G	591.6	81.7
RegNetY-16G [48]	224 ²	84M	16.0G	334.7	82.9
EffNet-B3 [58]	300 ²	12M	1.8G	732.1	81.6
EffNet-B4 [58]	380 ²	19M	4.2G	349.4	82.9
EffNet-B5 [58]	456 ²	30M	9.9G	169.1	83.6
EffNet-B6 [58]	528 ²	43M	19.0G	96.9	84.0
EffNet-B7 [58]	600 ²	66M	37.0G	55.1	84.3
ViT-B/16 [20]	384 ²	86M	55.4G	85.9	77.9
ViT-L/16 [20]	384 ²	307M	190.7G	27.3	76.5
DeiT-S [63]	224 ²	22M	4.6G	940.4	79.8
DeiT-B [63]	224 ²	86M	17.5G	292.3	81.8
DeiT-B [63]	384 ²	86M	55.4G	85.9	83.1
Swin-T	224 ²	29M	4.5G	755.2	81.3
Swin-S	224 ²	50M	8.7G	436.9	83.0
Swin-B	224 ²	88M	15.4G	278.1	83.5
Swin-B	384 ²	88M	47.0G	84.7	84.5

(b) ImageNet-22K pre-trained models					
method	image size	#param.	FLOPs	throughput (image / s)	ImageNet top-1 acc.
R-101x3 [38]	384 ²	388M	204.6G	-	84.4
R-152x4 [38]	480 ²	937M	840.5G	-	85.4
ViT-B/16 [20]	384 ²	86M	55.4G	85.9	84.0
ViT-L/16 [20]	384 ²	307M	190.7G	27.3	85.2
Swin-B	224 ²	88M	15.4G	278.1	85.2
Swin-B	384 ²	88M	47.0G	84.7	86.4
Swin-L	384 ²	197M	103.9G	42.1	87.3

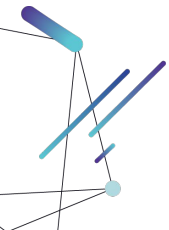
Table 1. Comparison of different backbones on ImageNet-1K classification. Throughput is measured using the GitHub repository of [68] and a V100 GPU, following [63].

Hierarchical Transformers

Идея иерархичности размера патчей кажется удачной для **dense visual tasks** (детекция/сегментация), и получила дальнейшее развитие:

- Pyramid Vision Transformer (PVT): <https://arxiv.org/pdf/2102.12122>
- Multiscale Vision Transformers (MViT): <https://arxiv.org/pdf/2104.11227>

Они отличаются подходами к тому, как сделать Attention более эффективным, и как реализовать иерархичность разрешения токенов.



Что мы уже рассмотрели

- **DeiT**: ViT с эффективной системой обучения и дистилляцией из CNN
- **Swin Transformer**: Vision Transformer с более вычислительно эффективным механизмом attention и идеей иерархичности представлений

Что если добавить в ViT идеи из CNN более явно, на архитектурном уровне?

ViT + CNN

CvT (Convolution Vision Transformer)

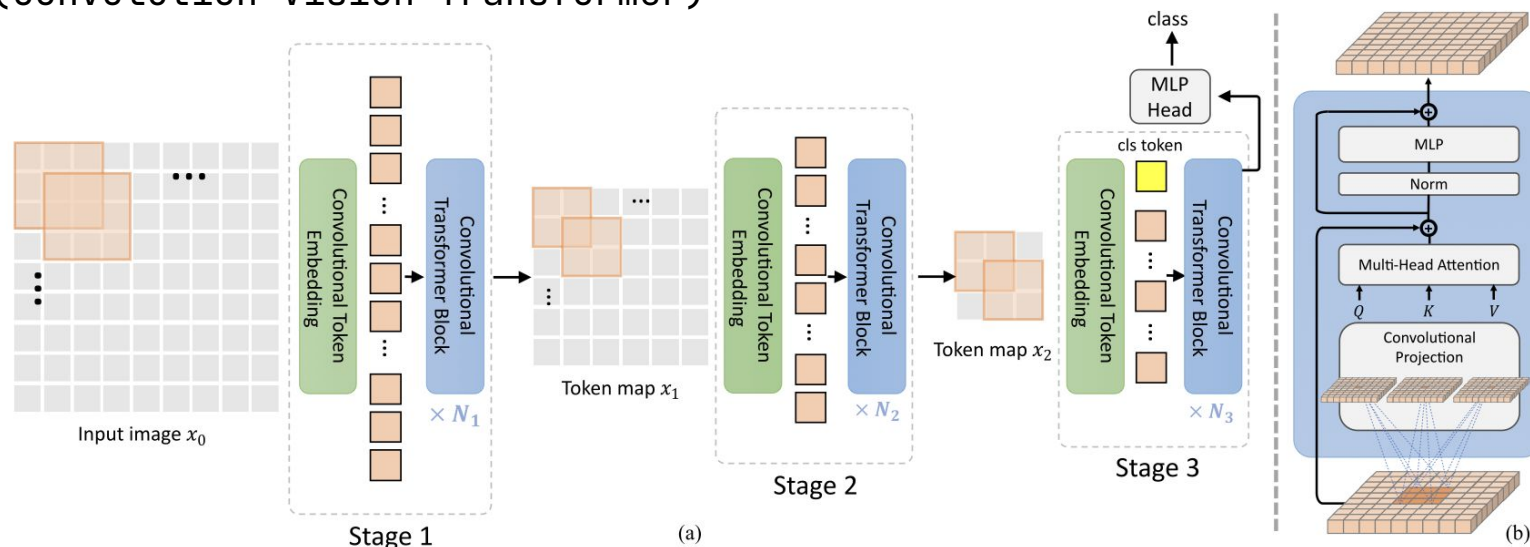
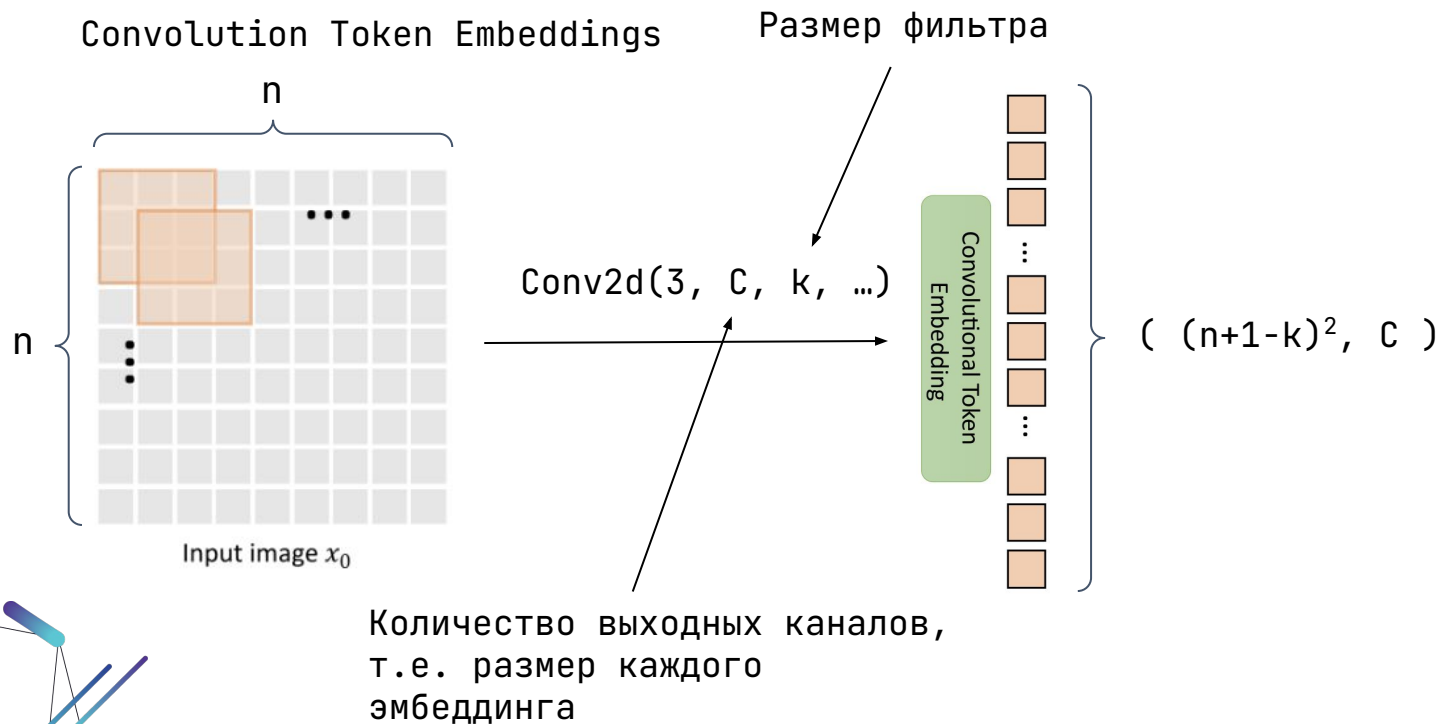
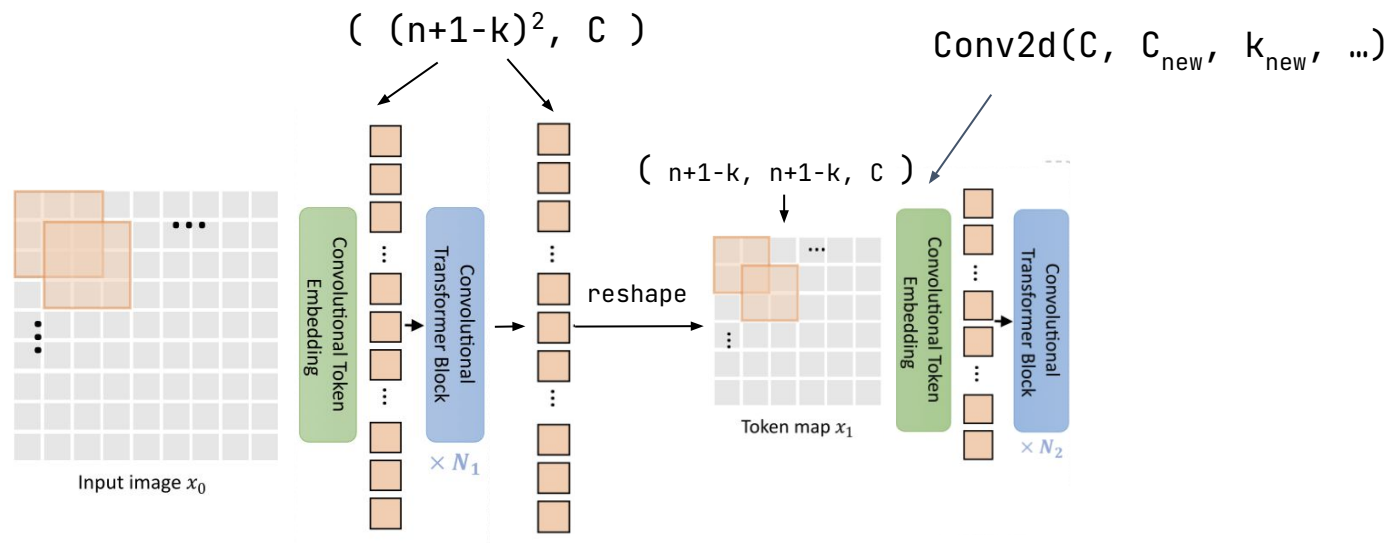


Figure 2: The pipeline of the proposed CvT architecture. (a) Overall architecture, showing the hierarchical multi-stage structure facilitated by the Convolutional Token Embedding layer. (b) Details of the Convolutional Transformer Block, which contains the convolution projection as the first layer.

ViT + CNN



ViT + CNN



ViT + CNN

CvT (Convolution Vision Transformer)

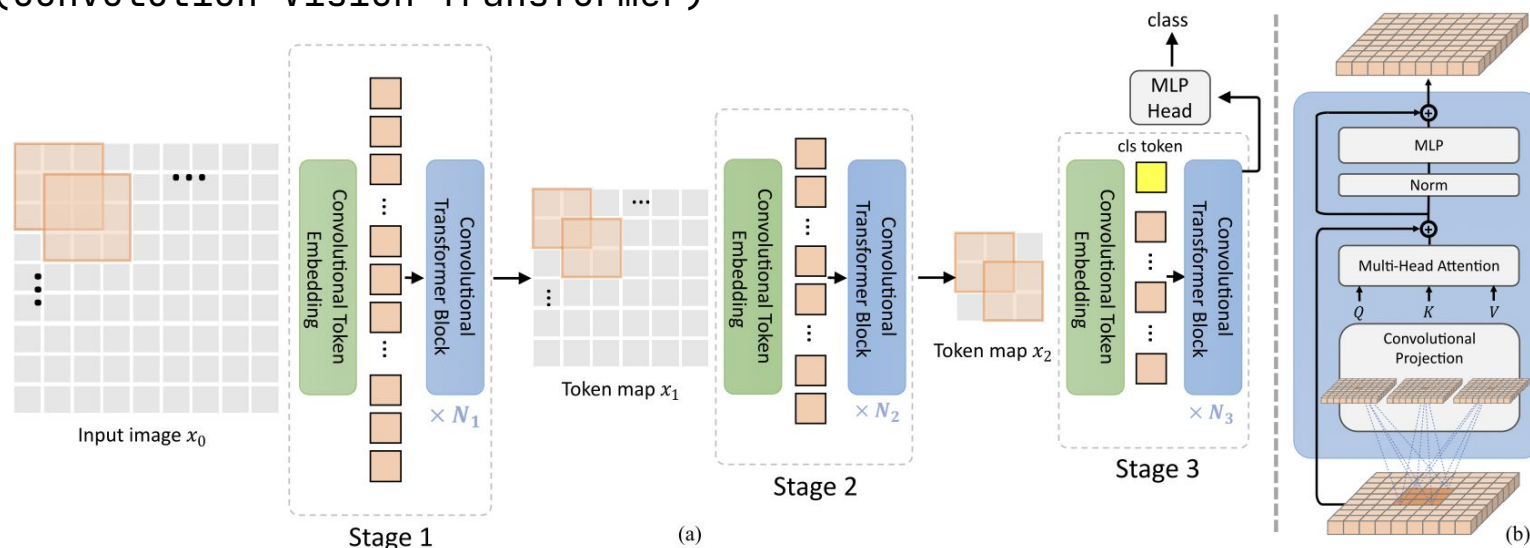
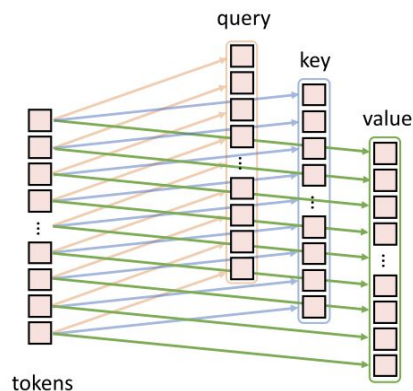


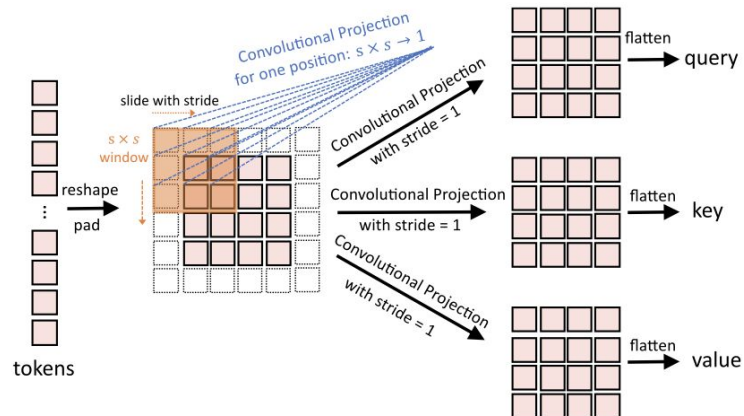
Figure 2: The pipeline of the proposed CvT architecture. (a) Overall architecture, showing the hierarchical multi-stage structure facilitated by the Convolutional Token Embedding layer. (b) Details of the Convolutional Transformer Block, which contains the convolution projection as the first layer.

ViT + CNN

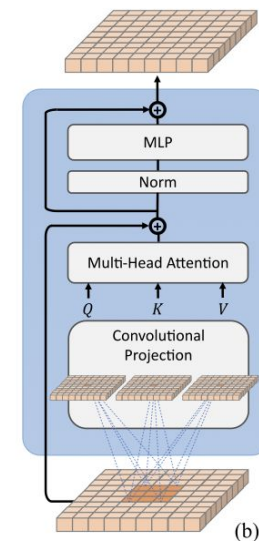
Convolutional Projection for Attention



Обычный attention

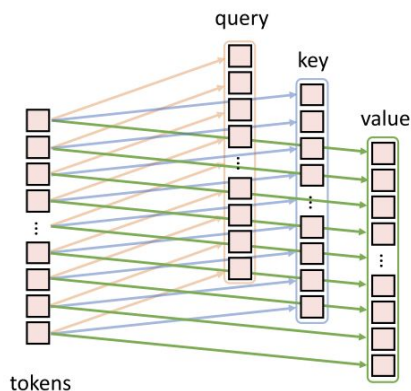


Attention в CvT

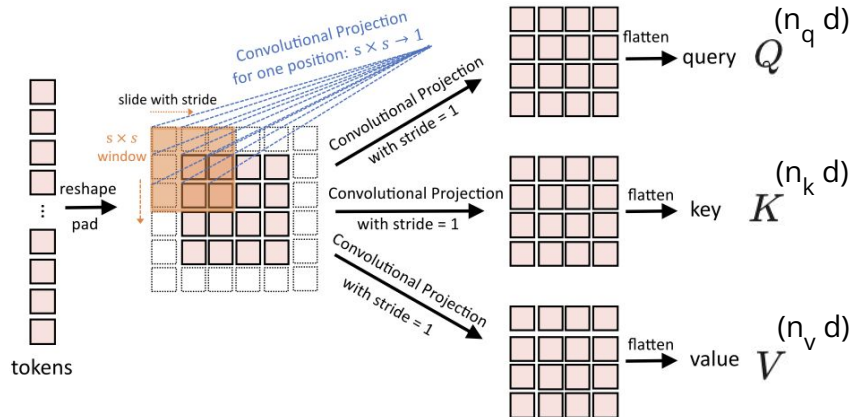


ViT + CNN

Convolutional Projection for Attention



Обычный attention



Attention в CvT

Подсчет весов
внимания:

$$A = \text{softmax} \left(\frac{QK^T}{\sqrt{d}} \right)$$

Размерность A:

$$(n_q d)(d n_k) = (n_q n_k)$$

Вычисление вектора
внимания:

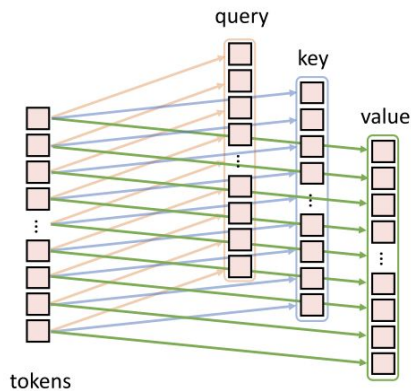
$$AV$$

Размерность AV:

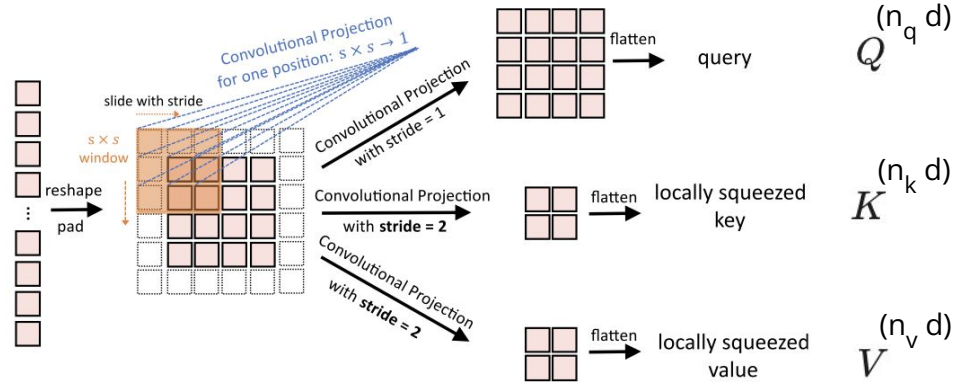
$$(n_q n_k)(n_v d) = (n_q d)$$

ViT + CNN

Convolutional Projection for Attention



Обычный attention



Attention в CvT

ViT + CNN

CvT (Convolution Vision Transformer)

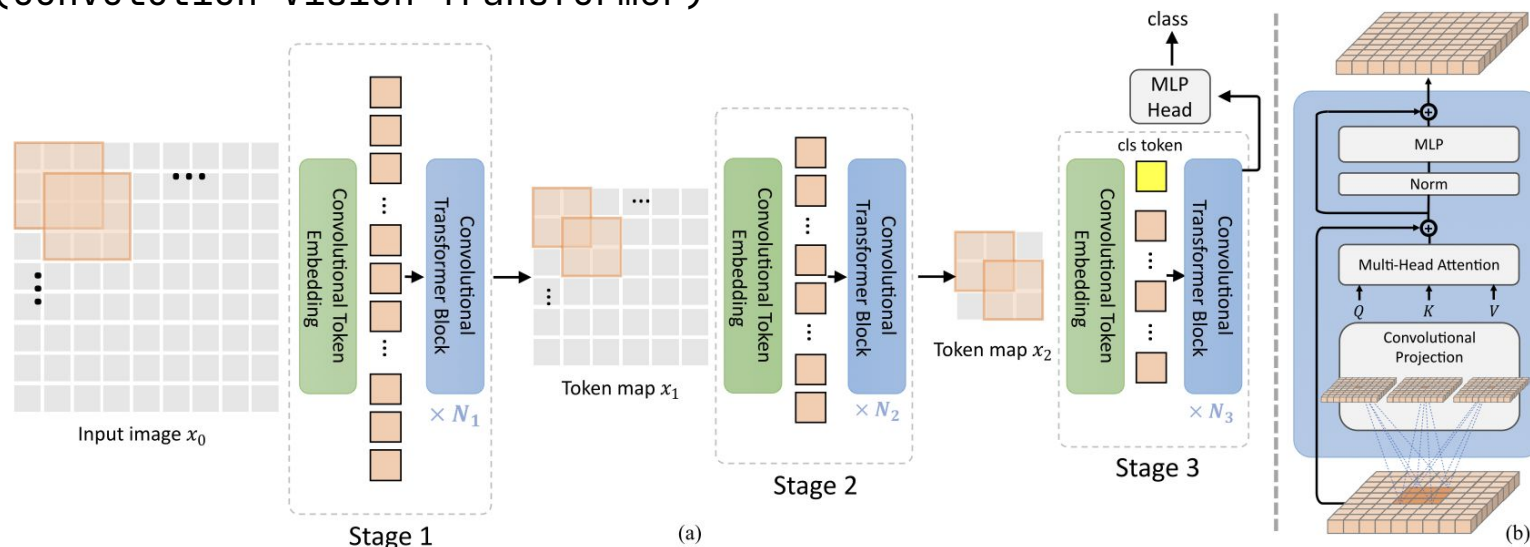


Figure 2: The pipeline of the proposed CvT architecture. (a) Overall architecture, showing the hierarchical multi-stage structure facilitated by the Convolutional Token Embedding layer. (b) Details of the Convolutional Transformer Block, which contains the convolution projection as the first layer.

Method Type	Network	#Param. (M)	image size	FLOPs (G)	ImageNet top-1 (%)	Real top-1 (%)	V2 top-1 (%)
<i>Convolutional Networks</i>	ResNet-50 [15]	25	224 ²	4.1	76.2	82.5	63.3
	ResNet-101 [15]	45	224 ²	7.9	77.4	83.7	65.7
	ResNet-152 [15]	60	224 ²	11	78.3	84.1	67.0
<i>Transformers</i>	ViT-B/16 [11]	86	384 ²	55.5	77.9	83.6	—
	ViT-L/16 [11]	307	384 ²	191.1	76.5	82.2	—
	DeiT-S [30][arxiv 2020]	22	224 ²	4.6	79.8	85.7	68.5
	DeiT-B [30][arxiv 2020]	86	224 ²	17.6	81.8	86.7	71.5
	PVT-Small [34][arxiv 2021]	25	224 ²	3.8	79.8	—	—
	PVT-Medium [34][arxiv 2021]	44	224 ²	6.7	81.2	—	—
	PVT-Large [34][arxiv 2021]	61	224 ²	9.8	81.7	—	—
	T2T-ViT _t -14 [41][arxiv 2021]	22	224 ²	6.1	80.7	—	—
	T2T-ViT _t -19 [41][arxiv 2021]	39	224 ²	9.8	81.4	—	—
	T2T-ViT _t -24 [41][arxiv 2021]	64	224 ²	15.0	82.2	—	—
	TNT-S [14][arxiv 2021]	24	224 ²	5.2	81.3	—	—
	TNT-B [14][arxiv 2021]	66	224 ²	14.1	82.8	—	—
<i>Convolutional Transformers</i>	Ours: CvT-13	20	224 ²	4.5	81.6	86.7	70.4
	Ours: CvT-21	32	224 ²	7.1	82.5	87.2	71.3
	Ours: CvT-13 _{↑384}	20	384 ²	16.3	83.0	87.9	71.9
	Ours: CvT-21 _{↑384}	32	384 ²	24.9	83.3	87.7	71.9
	Ours: CvT-13-NAS	18	224 ²	4.1	82.2	87.5	71.3
<i>Convolution Networks</i> _{22k}	BiT-M _{↑480} [18]	928	480 ²	837	85.4	—	—
<i>Transformers</i> _{22k}	ViT-B/16 _{↑384} [11]	86	384 ²	55.5	84.0	88.4	—
	ViT-L/16 _{↑384} [11]	307	384 ²	191.1	85.2	88.4	—
	ViT-H/16 _{↑384} [11]	632	384 ²	—	85.1	88.7	—
<i>Convolutional Transformers</i> _{22k}	Ours: CvT-13 _{↑384}	20	384 ²	16	83.3	88.7	72.9
	Ours: CvT-21 _{↑384}	32	384 ²	25	84.9	89.8	75.6
	Ours: CvT-W24 _{↑384}	277	384 ²	193.2	87.7	90.6	78.8

Table 3: Accuracy of manual designed architecture on ImageNet [9], ImageNet Real [2] and ImageNet V2 matched frequency [26]. Subscript _{22k} indicates the model pre-trained on ImageNet22k [9], and finetuned on ImageNet1k with the input size of 384×384 , except BiT-M [18] finetuned with input size of 480×480 .

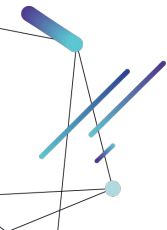
Еще варианты ViT + CNN

CoAtNet: Marrying Convolution and Attention for All Data Sizes
(<https://arxiv.org/pdf/2106.04803>)

Идея: скрестить механизмы attention и convolution kernel,
получив нужные inductive biases в одном механизме

Table 1: Desirable properties found in convolution or self-attention.

Properties	Convolution	Self-Attention
Translation Equivariance	✓	
Input-adaptive Weighting		✓
Global Receptive Field		✓



Еще варианты ViT + CNN

MaxViT: Multi-Axis Vision Transformer

(<https://arxiv.org/pdf/2204.01697>)

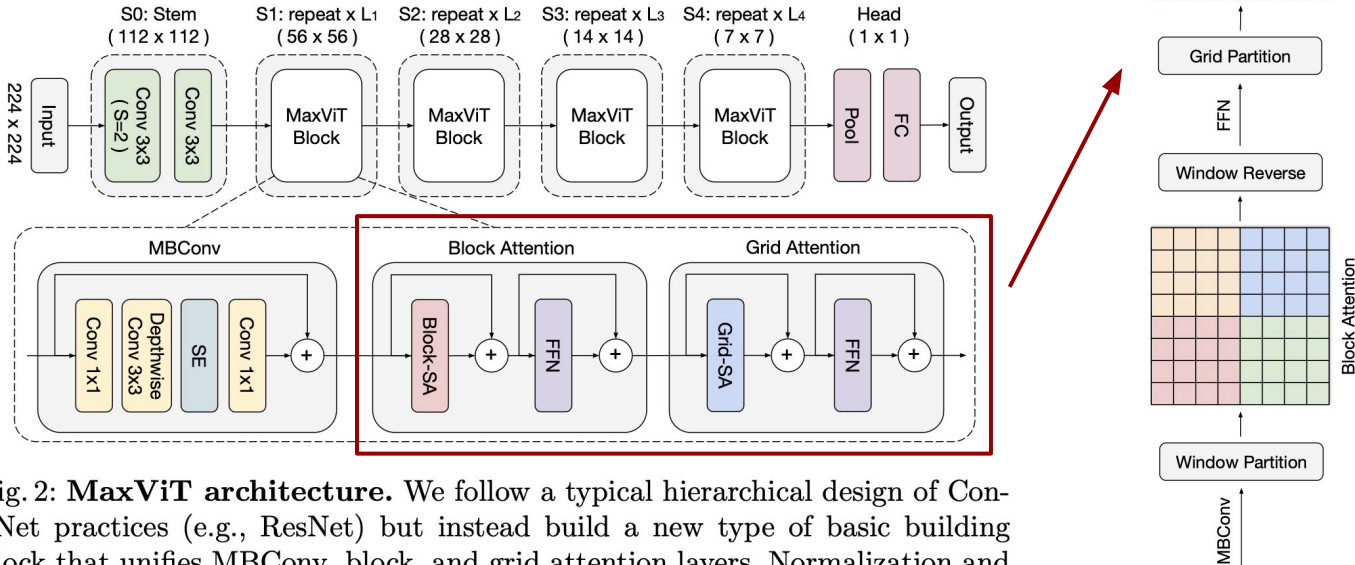
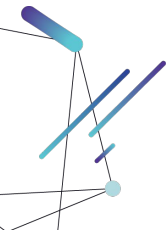


Fig. 2: **MaxViT architecture.** We follow a typical hierarchical design of ConvNet practices (e.g., ResNet) but instead build a new type of basic building block that unifies MBConv, block, and grid attention layers. Normalization and activation layers are omitted for simplicity.

Что мы уже рассмотрели

- **DeiT**: ViT с эффективной системой обучения и дистилляцией из CNN
- **Swin Transformer**: Vision Transformer с более вычислительно эффективным механизмом attention и идеей иерархичности представлений
- **CvT**: ViT + CNN на уровне архитектуры

Теперь будет идея DeiT, но наоборот: что если сделать CNN, в которую добавить идей из ViT?

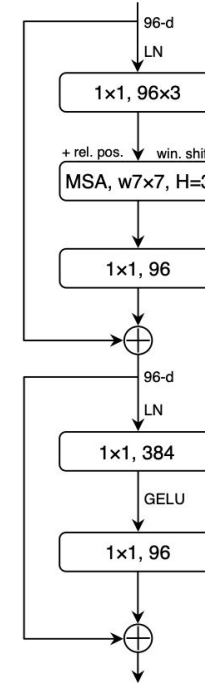


ConvNeXT

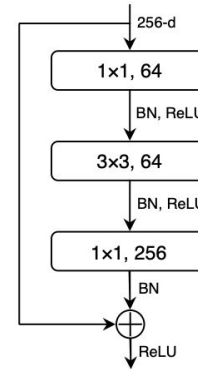
Это полностью сверточная сеть,
в которую перенесли некоторые
архитектурные решения из трансформеров:

- LayerNorm вместо batchNorm
- Фильтры большего размера (7x7)
как аналог self-attention
- Фильтры размера 1 как аналог MLP
- GeLU вместо reLU
- Меньше функций активаций

Swin Transformer Block



ResNet Block



ConvNeXt Block

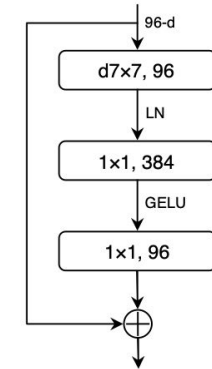


Figure 4. **Block designs** for a ResNet, a Swin Transformer, and a ConvNeXt. Swin Transformer's block is more sophisticated due to the presence of multiple specialized modules and two residual connections. For simplicity, we note the linear layers in Transformer MLP blocks also as "1x1 convs" since they are equivalent.

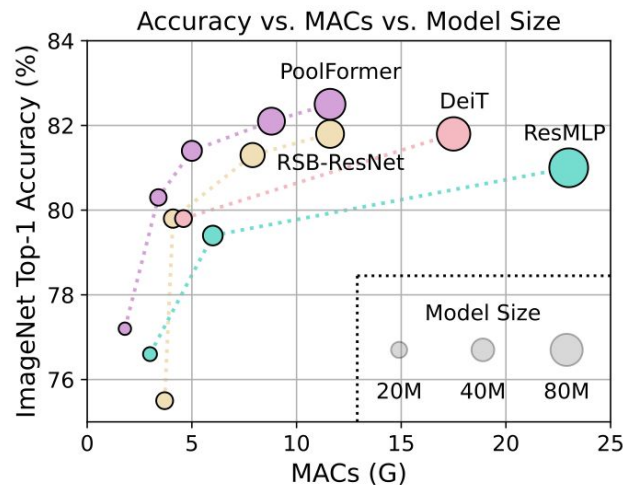
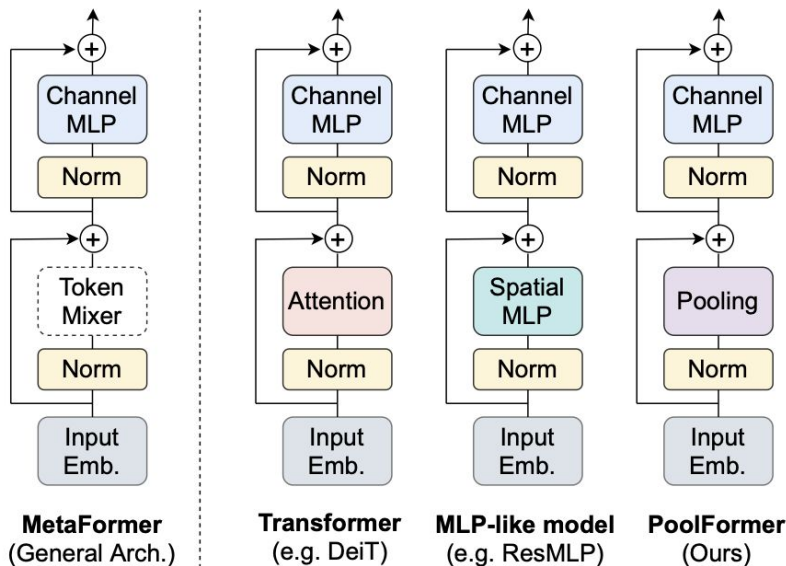
ConvNeXT

model	image size	#param.	FLOPs	throughput (image / s)	IN-1K top-1 acc.
ImageNet-1K trained models					
• RegNetY-16G [54]	224 ²	84M	16.0G	334.7	82.9
• EffNet-B7 [71]	600 ²	66M	37.0G	55.1	84.3
• EffNetV2-L [72]	480 ²	120M	53.0G	83.7	85.7
○ DeiT-S [73]	224 ²	22M	4.6G	978.5	79.8
○ DeiT-B [73]	224 ²	87M	17.6G	302.1	81.8
○ Swin-T	224 ²	28M	4.5G	757.9	81.3
• ConvNeXt-T	224 ²	29M	4.5G	774.7	82.1
○ Swin-S	224 ²	50M	8.7G	436.7	83.0
• ConvNeXt-S	224 ²	50M	8.7G	447.1	83.1
○ Swin-B	224 ²	88M	15.4G	286.6	83.5
• ConvNeXt-B	224 ²	89M	15.4G	292.1	83.8
○ Swin-B	384 ²	88M	47.1G	85.1	84.5
• ConvNeXt-B	384 ²	89M	45.0G	95.7	85.1
• ConvNeXt-L	224 ²	198M	34.4G	146.8	84.3
• ConvNeXt-L	384 ²	198M	101.0G	50.4	85.5

model	image size	#param.	FLOPs	throughput (image / s)	IN-1K top-1 acc.
ImageNet-22K pre-trained models					
• R-101x3 [39]	384 ²	388M	204.6G	-	84.4
• R-152x4 [39]	480 ²	937M	840.5G	-	85.4
• EffNetV2-L [72]	480 ²	120M	53.0G	83.7	86.8
• EffNetV2-XL [72]	480 ²	208M	94.0G	56.5	87.3
○ ViT-B/16 (👤) [67]	384 ²	87M	55.5G	93.1	85.4
○ ViT-L/16 (👤) [67]	384 ²	305M	191.1G	28.5	86.8
• ConvNeXt-T	224 ²	29M	4.5G	774.7	82.9
• ConvNeXt-T	384 ²	29M	13.1G	282.8	84.1
• ConvNeXt-S	224 ²	50M	8.7G	447.1	84.6
• ConvNeXt-S	384 ²	50M	25.5G	163.5	85.8
○ Swin-B	224 ²	88M	15.4G	286.6	85.2
• ConvNeXt-B	224 ²	89M	15.4G	292.1	85.8
○ Swin-B	384 ²	88M	47.0G	85.1	86.4
• ConvNeXt-B	384 ²	89M	45.1G	95.7	86.8
○ Swin-L	224 ²	197M	34.5G	145.0	86.3
• ConvNeXt-L	224 ²	198M	34.4G	146.8	86.6
○ Swin-L	384 ²	197M	103.9G	46.0	87.3
• ConvNeXt-L	384 ²	198M	101.0G	50.4	87.5
• ConvNeXt-XL	224 ²	350M	60.9G	89.3	87.0
• ConvNeXt-XL	384 ²	350M	179.0G	30.2	87.8

MetaFormer

Что если преимущество ViT не в механизме внимания, а в общем строении архитектуры?



PoolFormer

Pooling – AveragePooling внутри окна размера $k \times k$:

$$AvgPool(X_{i,j}) = \frac{1}{k^2} \sum_{(u,v) \in N_k(i,j)} X_{u,v}$$

$$Pooling(X) = AvgPool(X) - X$$

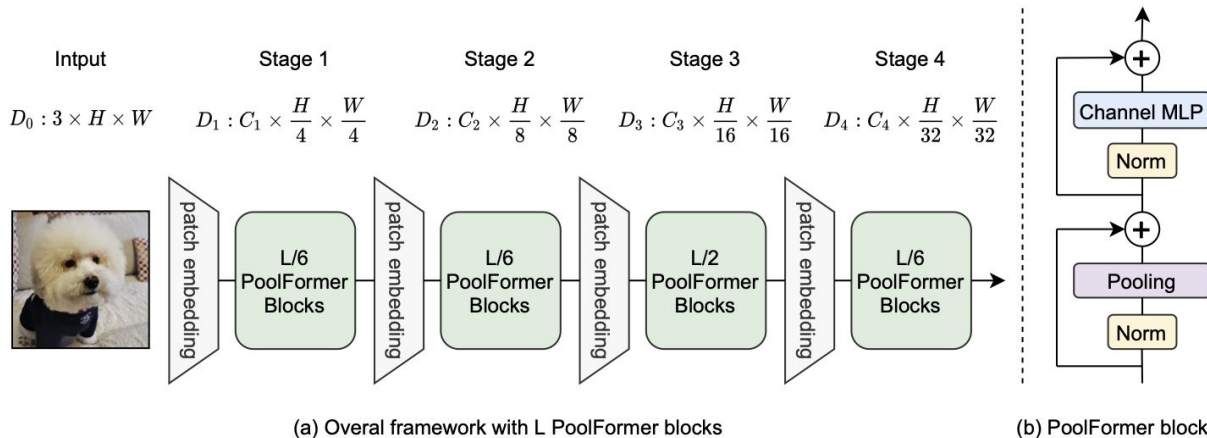
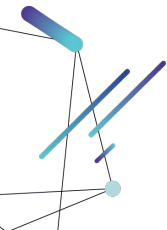


Figure 2. **(a) The overall framework of PoolFormer.** Similar to [24, 36, 57], PoolFormer adopts hierarchical architecture with 4 stages. For a model with L PoolFormer blocks, stage [1, 2, 3, 4] have $[L/6, L/6, L/2, L/6]$ blocks, respectively. The feature dimension D_i of stage i is shown in the figure. **(b) The architecture of PoolFormer block.** Compared with Transformer block, it replaces attention with extremely simple non-parametric operator, pooling, to conduct only basic token mixing.

Итоги

В этом видео мы изучили разные варианты архитектуры ViT, ViT + CNN, и разные варианты паттернов внимания:

- **DeiT**: ViT с эффективной системой обучения и дистилляцией из CNN
- **Swin Transformer** (... PVT, MViT): Vision Transformer с более вычислительно эффективным механизмом attention и идеей иерархичности представлений
- **CvT** (... CoAtNet, MaxViT): ViT + CNN на уровне архитектуры
- **ConvNeXT**: CNN с архитектурными изменениями, которые вдохновлены ViT
- **MetaFormer**: архитектура ViT, но attention заменен на TokenMixer (например, Pooling)



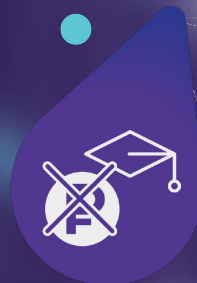


Deep Learning School

бесплатно.



онлайн.



фундаментально.